OPERATING FRAMEWORK / VERSION 1.0

AI Team Operating Handbook

From customer request to supported product

A practical framework for a developing data science team working with restricted networks and departmental data.

29 chapters | 24 operating procedures
Editable templates | Two Python dashboard examples

Prepared for James Amos
9 September 2026

Includes a required Claude.md code-summary standard.

Contents

Use this linked table of contents or the PDF bookmarks to jump to a chapter. Detailed procedures, templates and dashboard instructions are included in the body.

01 | How to use this operating handbook 3

02 | Ownership, roles, and decision rights 5

03 | Product lifecycle and review gates 7

04 | Customer intake, effort estimation, and priority handoff 9

05 | Product cards, registries, and authoritative records 11

06 | Data ownership, access, credentials, and lineage 13

07 | Restricted-network architecture and security design 16

08 | GitLab, Confluence, and the minimum shared platform 18

09 | Git and repository procedures for less experienced staff 20

10 | Required Claude.md and responsible coding-assistant use 22

11 | Standard development plan and software structure 25

12 | Data pipelines and data contracts 27

13 | Models, evaluation, and reproducibility 29

14 | Containerized LLM services and shared-model operations 31

15 | Testing, CI, packaging, and offline releases 33

16 | Delivery, acceptance, and support agreements 35

17 | Monitoring, health, model changes, and maintenance 37

18 | Incidents, changes, recovery, and retirement 39

19 | Team training, cadence, capacity, and lead responsibilities 41

20 | Standard operating procedure catalog 43

21 | Copyable templates and required records 50

22 | Dashboard A: project and product portfolio 53

23 | Dashboard B: data access and reliability 56

24 | Dashboard engineering plan and production hardening 59

25 | Worked examples from request to retirement 62

26 | Thirty-, sixty-, and ninety-day adoption plan 64

27 | Optional capability menu and selection triggers 65

28 | Glossary and quick reference 67

29 | Sources, version notes, and deliverable map 69

01 | How to use this operating handbook

This handbook is a proposed operating model for a data science team delivering multiple AI products inside a restricted network. It assumes uneven engineering experience, departmental ownership of data, approval-dependent access, GitLab and Confluence availability, limited internal coding assistants, and model services commonly running in server containers. It does not assume that new platforms, administrator access, internet connectivity, GPUs, or production support already exist.

The goal is to let the next person understand what a product does, why it exists, who owns it, where its data comes from, how to reproduce it, and what to do when it fails. A product can be a report, a script, a recurring batch process, a prediction API, or an interactive LLM application. A project is a time-bounded effort to create or change a product. Closing the project does not automatically end the product's operating obligations.

Adoption rule: start with the core controls below, then add the conditional practices that match the use case. Proposed cadences, thresholds, effort examples, and role assignments in this handbook are starting points for local agreement. They are not claims about your organization's policies, system capabilities, or actual staffing.

1.1 Read by role

Reader	Start here	Result
Lead data scientist	Chapters 2-5, 19, 25-27	Ownership, gates, capacity, adoption plan
New team member	Chapters 8-13, 20, 28	Repeatable daily workflow and vocabulary
Customer or business owner	Chapters 4-5, 16, 21	Clear request, scope, acceptance and support
Data owner or security partner	Chapters 6-7, 14-18, 24	Explicit permissions and operating boundaries
Dashboard developer	Chapters 22-24 and toolkit README	Two examples and the production build plan

1.2 Core, conditional, and optional practices

Level	Apply when	Practices
Core	Every code or analysis deliverable	Product ID and owner; customer POC; approved data purpose; access record; Git history; README; Claude.md; reproducible commands; review; validation evidence; acceptance; retention and support decision
Conditional	Repeated execution or multiple users	Service identity; scheduler; idempotent pipeline; deployment record; runbook; monitoring; backup; access renewal; incident route; maintenance capacity
Conditional	Sensitive data or consequential decisions	Security and data-owner review; stronger isolation; independent evaluation; human decision controls; audit; failure and misuse tests
Optional	A demonstrated scaling need exists	Workflow orchestrator; model registry; feature store; vector database; GPU scheduler; canary releases; automated drift analysis; high availability

Use a documented "not applicable" reason for a conditional artifact you do not need. Do not fabricate a service-level objective for an archived one-time report. Do not omit basic data authorization because an experiment is small.

1.3 First decisions to make

1. Name the lead, technical deputy, portfolio coordinator, and production-support contact.
2. Agree on the product ID format and the authoritative product registry.
3. Select one approved repository template and one work-item template.
4. Confirm the data-access request route with each source department.
5. Identify the permitted execution environments, model endpoints, and package-import process.
6. Choose the customer acceptance authority and the engineering release authority.
7. Define which products the team will operate and which require a receiving owner.
8. Reserve capacity for reviews, onboarding, and maintenance before promising new work.
9. Pilot this model on two cases: one small one-time delivery and one maintained product.
10. Review the process after the pilots and remove steps that have no decision or evidence value.

1.4 Scope and evidence

All examples, names, product IDs, thresholds, and records are synthetic. Public primary sources support specific technical notes and are listed in Chapter 29. The overall workflow, templates, dashboards, and illustrative service expectations are original design recommendations. Local security, records-management, procurement, and accreditation authorities determine which requirements apply.

02 | Ownership, roles, and decision rights

2.1 One named accountable owner for each decision

Assign people, not only departments. A department may sponsor a request, but a person must accept scope, answer questions, and resolve competing priorities. A small team can combine roles; preserve independent review for consequential changes. If no reviewer is available, arrange a qualified peer from another team before release.

Role	Accountable for	Minimum recorded information
Business owner	Value, scope, customer acceptance, operating funding	Name, department, deputy, acceptance authority
Customer POC	Daily questions, examples, user feedback	Name, contact route, availability
Technical owner	Design, implementation, validation, technical handoff	Name, deputy, repository and runbook
Data owner	Allowed purpose, approved users and processing conditions	Department, owner, approval route
Data custodian	Source connection, schema, provisioning, change notice	Technical POC and support route
Platform or service owner	Runtime, infrastructure, patching, availability	System owner, maintenance windows, escalation
Security or authorizing partner	Applicable controls, access boundaries, exceptions	Review route and decision record
Independent reviewer	Code and evidence review	Named reviewer and review scope
Portfolio/prioritization team	Relative business priority across cases	Decision, date, rationale, next review
Operations owner	Monitoring, incidents, renewal and retirement	Named primary, backup, support hours

The technical owner may recommend priority but does not silently become the prioritization authority. A customer approval does not authorize a deployment or data entitlement beyond that customer's authority.

2.2 Decision matrix

Decision	Accountable role	Work performed by	Required consultation
Define the business outcome	Business owner	POC and data scientist	Intended users
Estimate delivery effort	Technical owner	Delivery team	Data, platform and security partners
Assign portfolio priority	Prioritization team	Portfolio coordinator	Business and technical owners
Authorize data use	Data owner or designated authority	Custodian and access administrators	Security and records partners as applicable
Accept technical evidence	Technical owner	Independent reviewer	Domain expert
Accept business usefulness	Business owner	Customer testers	Intended users
Authorize operational release	Designated release authority	Operator or platform engineer	Technical, business and security owners
Retire a product	Business owner	Operations owner	Data owner and records partner

2.3 Ownership rules

- Every active product has a technical owner and a backup who can locate the runbook and execute a supported run.
- Every maintained product has a support owner, a support window, a maintenance allocation, and an escalation route.
- Every dataset has an owner and a custodian; the person who sent a sample is not automatically the owner.
- Every blocker has a next action, an action owner, a due date, and a last-contact date.
- Ownership changes require an explicit receiving-owner acknowledgment and a handoff checklist.
- Record decisions with their evidence and effective date. A dashboard field alone does not establish authorization.

2.4 When a role is missing

Keep discovery bounded if there is no customer POC. Pause data-dependent work if there is no valid access decision. Do not declare a maintained product operational without a support owner. Offer a smaller, explicitly accepted handoff when the organization can fund development but not ongoing operation. Record the constraint so it informs priority rather than becoming invisible overtime.

03 | Product lifecycle and review gates

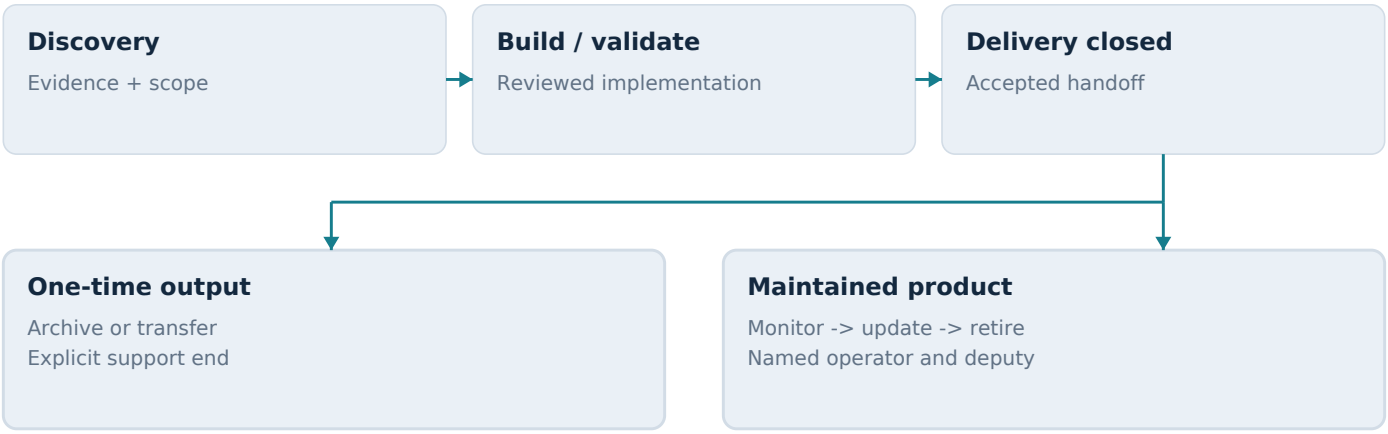
3.1 Track independent dimensions

A single status such as "in progress" cannot describe a portfolio. Keep delivery progress, live-service state, health, and blockers separate. A delivered product can be healthy and monitored while a new enhancement project is in build. A closed project can leave a supported product in service.

Dimension	Suggested values	Meaning
Delivery phase	Intake, Discovery, Ready, Build, Validation, Delivered, Closed, Cancelled	Progress of a delivery effort
Service state	Not live, Monitored, Maintenance only, Transferred, Retired	Current responsibility for the delivered product
Execution state	Active, Blocked, Paused	Whether work can proceed; retain the actual delivery phase
Health	Healthy, Degraded, Failed, Stale, Unknown, Not applicable	Current evidence about a live capability
Priority	Assigned rank or tier, Unranked	Decision owned by the prioritization team
Risk class	Low, Moderate, High, with rationale	Consequence and exposure; independent of difficulty

Use "Cancelled" for an abandoned delivery effort, "Retired" for an intentionally withdrawn product, and "Transferred" only after a receiving owner accepts it. Maintenance-only still needs a stated support policy. A blocked project is not automatically a low-priority project.

ONE PRODUCT, TWO SEPARATE TRACKS



A new enhancement can enter delivery while the existing product remains monitored.

3.2 Gates and evidence

Gate	Enter when	Evidence required to exit
G0: Request accepted for discovery	A request has an ID and POC	Problem, owner, expected value, first scope, initial data sources
G1: Feasibility assessed	Discovery has a bounded question	Representative sample or documented access gap; baseline; risk; low/likely/high estimate; uncertainty
G2: Ready to build	Prioritization and capacity permit	Agreed MVP; usable approved data for planned tasks; acceptance plan; dependencies and environment readiness
G3: Ready for customer validation	Working implementation exists	Reproducible build; review; technical tests; evaluation against baseline; known limitations
G4: Ready to release or hand off	Customer validation is complete	Customer acceptance; release authority; data permissions; artifact versions; runbook; support/retention decision
G5: Delivery closed	Transfer or operation is established	Receiving owner or operational owner acknowledgment; evidence archived; next review or retirement date
G6: Product retired	Retirement is approved	Jobs stopped; identities handled; consumers notified; records retained or disposed; registry updated

Gate results are pass, conditional pass, or fail. A conditional pass names the unmet condition, owner, limit, expiry, and applicable approval authority. It cannot waive an access control or security requirement on the team's own authority.

3.3 Transition rules

Discovery can return a case for clarification, recommend a short experiment, or recommend stopping. Ready can return to discovery after a material assumption changes. Validation can return to build after failed acceptance. Delivered or closed products can receive a new linked change project; production records should preserve the earlier release history. The educational dashboard permits simplified reopen transitions while its audit log retains prior values.

A change to users, data purpose, model endpoint, automation privileges, or deployment boundary triggers renewed assessment of the affected gates. A spelling correction in a user guide usually does not need a full model evaluation.

3.4 Definitions of ready and done

Ready for an individual work item: owner, outcome, acceptance condition, dependencies, data authorization, and estimated size are present. No item should be "build the AI" without a bounded result.

Done for an individual work item: reviewed change, relevant tests, linked evidence, updated documentation including Claude.md when affected, and a usable result. A notebook running on its author's machine is not sufficient evidence for a shared product.

04 | Customer intake, effort estimation, and priority handoff

4.1 Ranked customer questions

These preserve the intake structure from your customer-discovery questionnaire. Record examples, numbers, evidence quality, an answer owner, and unresolved questions.

Rank	Ask the customer
1	What specific business problem are you trying to solve, and who is affected?
2	What measurable benefit would solving it deliver: time, cost, quality, revenue, or risk?
3	How is the work performed today, and where does the process fall short?
4	What should the solution take as input and produce or do as output? Walk through a real example.
5	What results would make it successful enough to use?
6	What supporting data already exists, and where is it stored?
7	Can that data be accessed and used for this purpose, and who must approve access?
8	How much data exists, and how complete, consistent, and representative is it?
9	What happens if the answer or action is wrong, and where is human review required?
10	What is the smallest useful first version, and which capabilities can wait?
11	Do you have trusted examples, labels, or experts who can judge correctness?
12	Which applications, databases, or workflows must it integrate with?
13	What security, privacy, compliance, or organizational restrictions apply?
14	Where must it run, and what infrastructure or approved tools are available?
15	What users, volume, response times, and availability must it support?
16	Who owns the case, approves requirements, and accepts the result?
17	What deadline or event drives the timing, and what happens if it is missed?
18	What dependencies or unresolved issues could delay the work?
19	What funding and staff time can support development, expert feedback, testing, and rollout?
20	Who owns monitoring, maintenance, support, and costs after launch?

4.2 Conduct the interview

Before the meeting, ask for one permitted input/output example and an overview of the existing workflow. During the meeting, start with the problem and current process, then explore desired behavior and data. Translate technical terms: a "label" is a trusted answer; "latency" is how long the user waits. Distinguish desired features from requirements and fixed constraints.

Afterward, return a short discovery summary through the approved channel. Separate confirmed statements from estimates and unknowns. Ask the customer to correct the interpretation, identify the decision owner, and supply missing evidence. Create work items for engineering investigations and separate dependency records for external approvals.

4.3 Estimate work, not only model coding

Work package	Include	Typical source of uncertainty
Discovery and design	Interviews, sample review, baseline, architecture	Incomplete workflow and acceptance criteria
Access and data preparation	Requests, connectors, parsing, cleaning, labels	Owner response, schema changes, data quality
Implementation	Pipeline, model integration, UI/API, error handling	Endpoint capabilities and integrations
Validation	Evaluation set, human review, failure tests, user testing	Ground truth and reviewer availability
Deployment and handoff	Packaging, approvals, runbook, training, restore test	Platform ownership and release route
Operation	Monitoring, incidents, updates, renewals, retirement	Volume, service hours, change frequency

Estimate each package with optimistic O, most likely M, and pessimistic P person-days. Require O <= M <= P. If useful, a planning center is (O + 4M + P) / 6; this is a heuristic, not a confidence interval. Explain the assumptions behind the range. Do not convert it to a deadline without dependency sequencing and staff availability.

Illustrative example: discovery 2/3/5, data preparation 3/6/12, implementation 5/10/18, validation 3/6/10, release 2/3/5 person-days. Totals are 15/28/50, with a weighted center of 29.5 person-days. At 60% allocation, 29.5 effort-days consumes about 49.2 working days for one person before accounting for dependency timing. A source approval taking ten working days may overlap some tasks or extend the critical path; do not automatically add it twice.

Report hands-on effort, external wait, elapsed schedule, ongoing support, and estimate confidence separately. "We do not yet have a sample" is a reason for a feasibility investigation, not a precise fixed-price estimate. Re-estimate after access, baseline evaluation, and first integration.

4.4 Handoff to the prioritization team

Send the product card, value hypothesis and baseline, risk class, low/likely/high effort, dependency waits, readiness state, support cost, reuse opportunities, fixed deadlines, and unresolved decisions. Include evidence quality and the next review date. That team assigns relative priority and records its rationale. Do not turn technical readiness into an unauthorized business-priority score.

Useful portfolio measures are request-to-discovery time, access-wait time, active cycle time, blocked effort, accepted value, and maintenance burden. Avoid ranking individuals by lines of code or number of AI-generated changes.

05 | Product cards, registries, and authoritative records

5.1 Minimum product card

The card is the index to the product. Keep short fields structured and link detailed evidence. A registry may initially be reviewed YAML in a restricted GitLab repository and later become a database behind a dashboard. Choose one writer for each field.

Field group	Required fields	Review trigger
Identity	Product ID, title, problem, requesting department, creation date, product type	New request or scope change
People	Business owner, customer POC, technical owner, deputy, data owners, operator	Any reassignment
Outcome	Users, baseline, value hypothesis, MVP, exclusions, acceptance criteria	Discovery and change review
Lifecycle	Delivery phase, service state, blocker, health, board priority, next action	Weekly and on material events
Data	Dataset IDs, purpose, source departments, access-request IDs, allowed processing	New source or purpose
Engineering	Repository, runtime, model/service IDs, release, dependencies, evaluation	Each release
Operations	Support hours, runbook, monitors, renewals, retention, retirement date	Before release and recurring review
Evidence	Approval records, test results, decision log, last verification and verifier	Each gate

Do not put passwords, tokens, private keys, connection strings with credentials, sensitive example prompts, or raw records in a product card. Credential records hold a secret-manager reference and an owning team in the approved restricted system. Even group names, source paths, department relationships, and project titles can require restricted visibility.

5.2 Separate related records

Entity	Why it is separate	Key relationship
Product	Long-lived capability and operating responsibility	One product has many projects and releases
Project/change	Time-bounded delivery scope	Belongs to a product
Dataset	Source ownership and data contract	Many products can use many datasets
Access request/grant	Purpose, principal, scope, approval, expiry and verification	Links product, dataset and principal
Model service	Shared endpoint and runtime ownership	Used by several releases
Release	Immutable code/model/config/data references	One evaluated product version
Run/evaluation	Execution and evidence	References an exact release and dataset version
Incident/decision	Why action was taken	Links affected products and evidence

Many-to-many product/dataset links prevent copying the same source description into many cards. Grant scope must remain product- and purpose-specific even when a dataset is shared.

5.3 Product card example

AI-001: Document extraction pilot. Operations requests extraction of five fields from an approved document collection. Sponsor A owns business acceptance; Requester A is the POC; Engineer A is technical owner and Engineer B is deputy. Delivery phase is Build; service state is Not live. It is blocked on sample-access verification by the IAM team. The next action is a least-privilege connection test.

The proposed MVP exports structured records for human review. It excludes autonomous updates to the system of record. Acceptance uses a held-out sample covering document types and scan quality; the field-level threshold will be agreed after the baseline. Source DS-101 has a separate access record AR-001. The estimate is 12/20/34 person-days, explicitly synthetic. Runtime, retention, and operational owner must be agreed before release.

5.4 Keep the card trustworthy

Store created_at, updated_at, updated_by, record_version, and evidence_checked_at. Require owners to acknowledge their records at an agreed cadence. Mark stale cards visibly. Changes to ownership, lifecycle, scope, permissions metadata, or support commitments produce an audit event. A dashboard should not silently overwrite a manual judgment with an imported field.

06 | Data ownership, access, credentials, and lineage

6.1 Separate four questions

1. **Authorization:** is this person or service allowed to use the data for this purpose?
2. **Authentication:** how does the source verify that identity?
3. **Connectivity:** can the approved workload reach the source?
4. **Usability:** does the permitted data contain the fields, history, and quality needed?

A working connection answers neither authorization nor fitness for purpose. Approval does not prove provisioning is complete. A developer's interactive permission does not imply that a scheduled service account may run the same query.

6.2 Access-request record

Record element	What to capture
Request identity	Request ID, product ID, dataset ID, requesting department, purpose
Principal	Named human or service identity reference; owning team; environment
Scope	Approved tables, fields, rows, operations, network path, output restrictions
Decision chain	Data owner, custodian, security partner if needed, tickets and evidence
Provisioning	Group/role reference, administrator, provisioned date, secret-store reference if applicable
Verification	Allowed-operation test, denied-operation test, tester, time, result
Lifecycle	Requested, owner review, security review if needed, approved, provisioned, verified, denied, expired, revoked
Renewal	Expiry, review date, renewal owner, notification date, revocation route
Derivatives	Approved staging, extracts, embeddings, indexes, logs, backups and outputs

Record approval and provisioning milestones separately even if the UI shows a summarized status. A denial should retain rationale and possible smaller scopes. Do not change a denied request to approved merely because another user can access the same dataset.

6.3 Data-access SOP

Trigger: a new source, user group, processing purpose, deployment environment, or service principal.
Owner: technical owner coordinates; the designated data authority decides.

1. Describe the business purpose and smallest required data scope. Identify the source owner and custodian.
2. Confirm classification/handling labels, permitted users, approved environments, retention, export limits, and derivative-data conditions.
3. Submit the access request through the source department's accepted workflow. Link its reference in the registry.
4. Request human and service identities separately. Prefer group-based entitlements and least privilege; do not borrow credentials.

5. Have the responsible administrators provision network, database, and secret-management access.
6. Test from the actual execution environment. Demonstrate one permitted read and one prohibited action or scope; protect test evidence.
7. Record verification and expiry. Begin only the processing authorized by the grant.
8. Schedule renewal and periodic review. On expiry or revocation, stop affected processing and apply the agreed handling of cached and derived material.

Evidence: approval, principal/scope mapping, connection test, denied-action result, expiry, and responsible owner. **Escalation:** overdue decisions go to the named departmental owner through the agreed route; the lead may rescope or resequence work. No shared login is a substitute.

6.4 Access matrix example

Principal type	Source read	Pipeline execution	Product metadata	Raw output/export
Customer user	Only if separately entitled	Usually request-only	Authorized products only	Only approved output scope
Data scientist	Approved development slice	Development environment	Assigned products	Approved evaluation scope
Service identity	Narrow production scope	Its scheduled task	Sanitized status writes	Approved destination only
Portfolio reader	No implied source access	No	Approved metadata projection	No implied export rights
Operator	Scoped operational access	Approved restart/release actions	Supported products	Only as required by runbook

This is an example policy design, not a grant. Actual rules may need attributes such as department, project membership, environment, data category, purpose, and time window in addition to roles.

6.5 Credentials and operational handling

Store secret values only in the approved secret-management mechanism. Retrieve them at runtime with a scoped identity. Use protected credential files with restrictive permissions only if that is the approved local method; define rotation and cleanup. Never commit secrets, paste them into an LLM, write them into command history, or include them in an exception trace.

Record a credential's owner, reference, scope, rotation date, expiry, and consumers in a restricted inventory. Avoid putting live secrets in command-line arguments. Coordinate rotation with a test and rollback path; rollback must not restore a revoked entitlement.

On departure or role change, disable access through the authoritative identity system, revoke relevant tokens, rotate shared secrets where applicable, and transfer product responsibility. Review service accounts for an owning team and active consumers.

6.6 Derived data, retrieval, and revocation

Extracts, training data, evaluation sets, embeddings, vector indexes, prompts, cached answers, and backups can inherit source restrictions. Record lineage from dataset version to transformations, model/run, output, and approved destination. Where data owners require it, propagate deletions and access revocations to derivatives on a defined schedule.

For retrieval-augmented generation, filter candidate documents using the current user's entitlements before sending content to the model; apply corresponding rules to citations, snippets and exports. An application's broad service account must not make every user's source visibility equally broad. Test revocation against previously indexed documents and cached answers. Stop or restrict retrieval if entitlement synchronization is stale under the agreed policy.

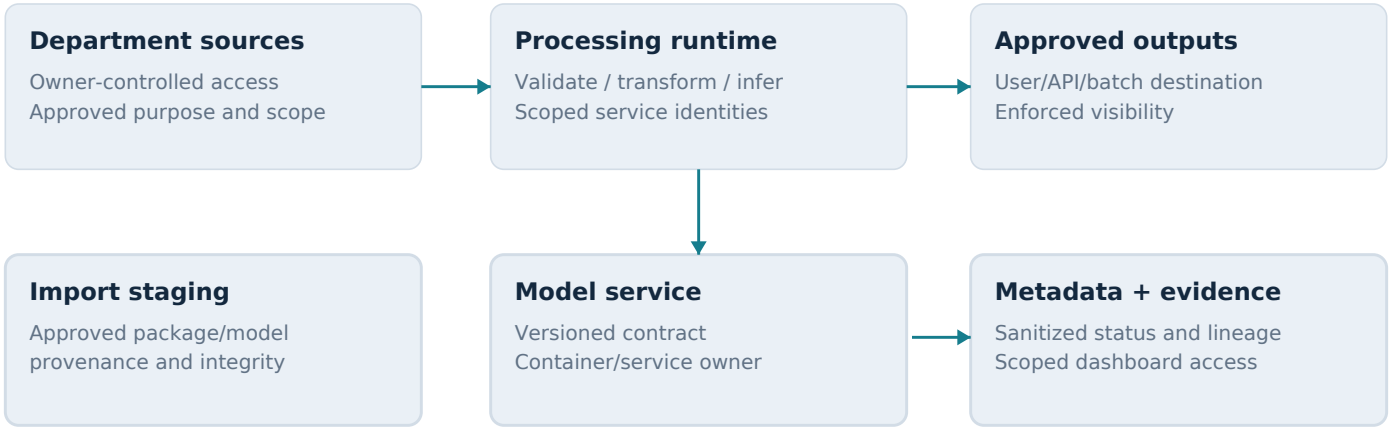
07 | Restricted-network architecture and security design

7.1 Discover the actual boundary

"Somewhat air gapped" can mean disconnected hosts, a restricted enclave, a mediated import path, or internal services with controlled upstream connectivity. Inventory actual network paths. Identify DNS, time synchronization, internal certificate authorities, identity provider, package/container registries, storage, logging, model endpoints, and administrator routes.

The presence of an internal chat interface or a tool called Claude does not establish where inference occurs. Ask the service owner which component executes inference, whether requests can leave the boundary, what content is logged, retention, supported model IDs, and the approved data categories. Record the answer in the model-service card.

LOGICAL BOUNDARIES TO VERIFY WITH LOCAL OWNERS



Identity, network policy, certificates, logging and recovery span these components.

Physical separation and connectivity depend on the approved local architecture.

7.2 Logical separation

Area	Purpose	Boundary rule
Departmental sources	Authoritative data and access policy	Data owner controls permissions and change notices
Development and evaluation	Bounded experiments and tests	Approved samples; scoped identities; separate outputs
Production runtime	Supported batch jobs and services	Approved release; service identities; operational controls
Metadata and evidence	Portfolio records, sanitized status and approvals	Visibility appropriate to metadata sensitivity
Approved import staging	Packages, images, documentation and model artifacts	Authorized transfer, provenance checks, local validation

These may be different servers, namespaces, directories, or enclaves depending on the local architecture. A diagram is not an implemented boundary. Containers package processes; they do not by themselves establish all required host, network, and data isolation.

7.3 Security review checklist by boundary

- Identify all data flows, trust boundaries, human identities, service identities, and external dependencies.
- List read, write, export, and administrative operations separately.
- Minimize raw-data movement; consider source-side queries or approved extracts when copying is restricted.
- Confirm encryption and certificate validation with the infrastructure owner; use the internal trust chain.
- Define logging fields and redaction before enabling verbose model or pipeline traces.
- Isolate development from production credentials, runtime writes, and model-service privileges.
- Treat third-party code, model files, document parsers, and imported containers as supply-chain inputs.
- Include unauthorized-access tests, unsafe-output handling, and denial/failure behavior in acceptance.

7.4 Import and dependency procedure

1. Specify the package/image/model version, platform, intended use, license, provenance, and destination.
2. Resolve dependencies in an authorized environment that matches the target Python, OS, CPU and accelerator requirements.
3. Assemble an approved bundle with exact versions, checksums, an artifact inventory or SBOM, scan results and transfer evidence.
4. Use the organization's transfer process. Preserve chain-of-custody and integrity records.
5. Recheck integrity inside the destination boundary; install into staging without external downloads.
6. Run functional and security checks, then promote the verified bundle to the internal repository.
7. Retain the previous supported bundle for rollback and track the next review or patch date.

pip supports repeatable installation using pinned requirements, hashes and local wheel bundles; compiled wheels can be platform-specific. The handbook combines these capabilities with a proposed organizational approval process. [R5]

Offline checksums establish file integrity against the recorded manifest, not that an artifact is trustworthy. Offline vulnerability results are only as current as the imported advisory database. Record its date and apply the local exception route when coverage is incomplete.

7.5 Continuity prerequisites

Make internal time, certificates, name resolution, identity services, storage capacity, backups, and renewal ownership visible dependencies. A service can fail because a certificate expires even when the model and code never change. Recovery instructions must be available when the primary dashboard or Confluence is unavailable.

08 | GitLab, Confluence, and the minimum shared platform

8.1 Keep Confluence and add a focused dashboard

Keep Confluence as the team's narrative knowledge base because it is already an organizational standard. Use the dashboard for structured records, current state, filters, and operational evidence. Use GitLab for versioned implementation and reviewed changes. Replacing Confluence would create a separate search, permissions, history, editing, and records-management product to maintain.

Information	Authoritative location	What other tools show
Code, tests, pipeline definitions, config schemas	Product GitLab repository	Links to commit/release
README, Claude.md, version-specific runbook	Product GitLab repository	Confluence overview and links
Business narrative, meeting decisions, onboarding	Confluence	Summary and canonical page link
Current product ownership and lifecycle fields	Chosen product registry	Dashboard view; link from Confluence
Work items, defects, merge review	GitLab issues and merge requests	Dashboard counts/status with freshness
Source approvals and entitlements	Owning department's approved system	Restricted reference and verified status
Secret values	Approved secret mechanism	Reference only in authorized inventory
Models, datasets and large artifacts	Approved artifact/data storage	Immutable IDs, checksums and locations
Historical run and monitoring evidence	Approved operational/evidence store	Sanitized metrics and drill-down links

If the registry starts as Git-reviewed YAML, make the first dashboard read-only. If later moving to database-backed editing, migrate once, record the cutover, and stop treating the old YAML as an independent writer. Keep a generated read-only snapshot if needed.

Confluence has distinct global, space and page-permission mechanisms. Configure the actual installed edition with its administrator and test access using representative accounts. A dashboard link, embedded view, or copied attachment must not widen the source audience. [R9]

8.2 Starter shared services

Establish a GitLab group with a repository template, a restricted metadata registry, an internal package/image source or approved offline bundle, a shared documentation home, and a known execution host. Add a self-managed CI runner only when a platform owner can patch, isolate and support it.

Create logical directories for approved raw inputs, validated intermediate data, released outputs, evaluation evidence, and runtime logs. Apply permissions and retention by dataset and product. Shared storage does not mean all staff receive access to all directories.

First automation: a standard command that validates configuration, checks input quality, runs a bounded job, and writes a manifest. Add scheduling only after that command is reliable. A single-host scheduler can support an initial recurring batch; an orchestrator becomes useful when dependencies, retries, backfills and concurrency exceed simple management.

8.3 Suggested Confluence structure

- Team home: mission, contacts, escalation, onboarding route, dashboard link.
- Operating handbook: approved procedures, templates, process owner, revision history.
- Product directory: one page per product, linked to its authoritative card and repository.
- Architecture decisions: problem, options, chosen approach, owner and consequences.
- Source directory: approved descriptions and request routes at the permitted visibility.
- Training and troubleshooting: practical lessons, common failures, internal examples.

Record an owner and review date on each durable page. Put runtime commands next to the versioned code; link them from Confluence to reduce drift.

09 | Git and repository procedures for less experienced staff

9.1 Plain-language Git workflow

Git records changes to files. A repository is the project history. A branch is a place to make a change without immediately changing the shared main branch. A commit records a focused change. A merge request asks another person to review and integrate that change. GitLab hosts the repository, work items and reviews.

Team default: short-lived feature branches, focused commits, one merge request per coherent change, independent review, and a protected main branch. Avoid a complicated branching model until release needs justify it. Configure available approval controls with the administrator; required approval features depend on GitLab tier and configuration. Where enforcement is unavailable, use restricted merge rights plus an explicit recorded review procedure. [R4]

9.2 New contributor procedure

1. Obtain your own GitLab access and approved authentication method.
2. Clone the repository from its internal GitLab address.
3. Read README, Claude.md, the product card, and the current issue.
4. Create the approved local environment using the internal package source.
5. Run the smallest supplied test or synthetic example before changing anything.
6. Create a feature branch linked to an issue.
7. Make a small change, run relevant checks, and inspect the diff.
8. Commit only intended files, push your branch, and open a merge request.
9. Address review feedback, update affected documentation, and let the authorized person merge.

```
git clone <approved-internal-repository-address>
cd <repository-directory>
git switch main
git pull --ff-only
git switch -c feature/AI-001-validate-input
git status
# Edit a small, issue-scoped change and run its checks.
git diff
git add src/validation.py tests/test_validation.py Claude.md
git diff --staged
git commit -m "AI-001 validate required input fields"
git push -u origin feature/AI-001-validate-input
```

Angle-bracket values are placeholders to replace before running. The named source files illustrate a product repository; they are not files in the dashboard demo. Prefer explicit file paths when staging. Verify that data, credentials, large models, generated caches, and local configuration have not entered the diff.

9.3 Merge-request checklist

- State the problem, change, resulting behavior, and linked issue.
- Identify data handling, interface, model, dependency, and configuration changes.
- Include meaningful test/evaluation results and known limitations.
- Update README, runbook, product metadata, and Claude.md where affected.

- If Claude.md needs no change, state why its summary remains correct.
- Confirm no raw restricted data, secrets, uncontrolled downloads or unsafe logging.
- Get a qualified human reviewer; model-generated approval does not replace review.
- Record customer or release approvals when this change crosses a gate.

9.4 Starter repository layout

Path	Purpose
README.md	Purpose, setup, run, test, limitations and owner
Claude.md	Required code summary for people and coding assistants
.claude/CLAUDE.md	Imports the requested summary using Claude Code's recognized convention
pyproject.toml or requirements inputs	Package metadata and dependency specification
requirements.lock and artifact manifest	Exact approved release dependency set and integrity information
src/	Reusable application and pipeline modules
tests/ and tests/fixtures/	Behavioral tests and synthetic/approved fixtures
configs/	Schemas and non-secret examples
notebooks/	Exploration with no hidden production responsibility
docs/	Design, evaluation plan, versioned runbook and decisions
.gitlab/	Issue and merge-request templates
.gitlab-ci.yml	Versioned automated checks
product.yaml	Product ID and approved registry references or initial canonical card

Notebook code should become a tested module before it is scheduled or reused by others. Clear sensitive outputs before committing, and keep large data outside Git. Document data snapshot references rather than copying datasets into the repository.

9.5 Common recovery situations

If a pull reports divergence, stop and ask a reviewer to help reconcile the branches; do not force-push main. For a conflict, understand both changes, resolve deliberately, and rerun the affected checks. For an incorrect merged change, use a reviewed revert where appropriate rather than rewriting shared history.

If a secret is committed, treat it as exposed: notify the responsible security/credential owner, revoke or rotate it, and follow the approved history-cleanup procedure. Deleting the visible line does not remove prior copies.

10 | Required Claude.md and responsible coding-assistant use

10.1 Exact filename convention

Every repository containing developed code must include **Claude.md** at its root. Its purpose is a concise, accurate summary of the code, interfaces, commands, dependencies, assumptions, and known limitations. Independently runnable subprojects in a monorepo should have their own scoped summary or a clearly indexed section with an accountable owner.

Claude Code documents automatic project instruction files named **CLAUDE.md**, including **.claude/CLAUDE.md**, and supports relative imports. To preserve your requested filename without placing two case-only names in the same directory, use the following loader. Verify behavior against your installed, possibly older internal version. [R10]

File: .claude/CLAUDE.md

```
@../Claude.md

Keep the code summary current with material changes.
```

If the installed tool does not support the loader convention, have the administrator select its supported setup and explicitly provide Claude.md as context in the meantime. The filename requirement remains part of human review.

10.2 Copyable code-summary template

```
# Code summary: <product ID and title>

## Purpose
Business task; intended users; supported and excluded behavior.

## Ownership
Technical owner; deputy; customer owner; support route.

## Code map
Entry points; important modules; where tests/configuration live.

## Data flow
Inputs -> validation -> transformations -> model/service -> outputs.
Dataset IDs and approved destinations; no raw examples or credentials.

## Interfaces and dependencies
Input/output schemas; internal service names; model versions;
runtime/Python requirements; approved dependency-lock reference.

## Run and test
Exact setup, synthetic example, test, evaluation, and startup commands.
Expected result; required permission references; no secret values.

## Configuration and security
Non-secret options; secret-manager reference pattern;
allowed network paths; logging/redaction; access assumptions.

## Operations
Runbook; monitoring; scheduling; release/rollback references.

## Limitations and unfinished work
Known failure modes; unsupported inputs; evidence gaps.

## Change discipline
Update this summary in the same merge request when it changes.
Record a no-change rationale when the summary remains accurate.
```

Keep the summary short enough to remain useful; put long procedures in versioned documentation and link them. A summary can describe a control, but filesystem, network, identity, application and CI controls must enforce it.

10.3 Coding-assistant procedure

Before use: confirm that the specific tool, endpoint, repositories and data categories are approved. Clarify tool permissions, file access, automatic memory/logging, extensions, network behavior, and retention. Limited or older models may need smaller tasks and more explicit context.

During work: give a bounded issue, the relevant interface, an example using synthetic data, and acceptance conditions. Ask for small changes that the author can explain. Review new dependencies, shell commands, data movement and generated configuration before executing them. Keep destructive or privileged actions within the organization's approval workflow.

After work: inspect every changed file, run relevant tests, check security assumptions, update Claude.md, and obtain independent human review. Record model/tool provenance where it is useful for reproducibility or required by local policy; a chat transcript is not a substitute for an engineering decision.

10.4 Suggested prompt pattern

```
Product: <ID>. Task: <one bounded behavior>.
Read Claude.md and the relevant interface before editing.
Inputs: <synthetic example or schema>.
Acceptance: <observable result and important failure behavior>.
Constraints: <approved libraries, no external calls, allowed files>.
Explain changed behavior and list checks actually run.
Update Claude.md if the code summary changes.
```

Do not assume an AI assistant can judge correctness in the customer's domain. Treat retrieved files and documents as untrusted input when building LLM products. Prompt instructions alone do not enforce tool permissions; limit capabilities in the application and runtime. OWASP describes prompt injection as a risk involving adversarial instructions within model inputs. [R2]

11 | Standard development plan and software structure

11.1 A complete development plan

Plan section	Required content
Problem and users	Workflow, baseline, value, sponsor, POC, intended operators
Scope	MVP, exclusions, assumptions, consequential decisions, human review
Data and access	Dataset contracts, approval references, representative samples, labels
Design	Components, interfaces, trust boundaries, dependencies, failure behavior
Work breakdown	Tasks, owners, low/likely/high effort, waits, review and training time
Validation	Baseline, frozen evaluation set, metrics, thresholds, customer test plan
Delivery	Runtime, packaging, release authority, runbook, transfer/training
Operations	Monitors, support, renewal, refresh, capacity, backup and retirement
Risks and decisions	Owner, probability/consequence, mitigation, evidence, next review

Plan investigations explicitly. A short feasibility task might answer whether an approved model can extract a field from representative documents. Its output is measured evidence and a recommendation, not an implied promise to productionize the experiment.

11.2 Suggested implementation sequence

1. Create the repository, product card, issue, README and Claude.md.
2. Make a synthetic input and a deterministic baseline run end to end.
3. Validate approved real samples and document data failures.
4. Connect the model/service behind an adapter with a stable output schema.
5. Add evaluation and failure handling before expanding functionality.
6. Add the user/API interface and source integrations incrementally.
7. Package and run from a clean environment using documented commands.
8. Conduct customer validation, operational rehearsal, and controlled handoff.

This sequence exposes missing data and environment assumptions early. It also gives junior staff a working path they can extend rather than an empty architecture diagram.

11.3 Modular boundaries

Module	Responsibility	Avoid placing here
Source adapter	Approved source queries and data retrieval	Model-specific prompt logic
Validation	Schema, type, range, required-field and data-quality checks	Silent deletion of bad records
Transformation	Deterministic cleaning/chunking/features	Hidden network calls
Model adapter	Inference request/response and capability checks	Business priority decisions
Evaluation	Compare predictions to trusted outcomes	Training on the held-out test answers
Application	Workflow, user interaction, human review	Secrets embedded in UI state
Orchestration	Dependencies, retries, scheduling and manifests	Duplicated business transformations
Observability	Sanitized logs, metrics and run identifiers	Raw sensitive payloads by default

Use explicit input/output schemas, dependency injection where useful, and configuration validated at startup. Keep paths, model IDs, timeout limits and thresholds configurable; keep secret values out of ordinary configuration. Favor simple functions and clear error classes over deep framework abstractions that the team cannot maintain.

11.4 Error and configuration contract

Classify failures as invalid input, access denied/expired, dependency unavailable, model timeout, invalid model output, resource exhaustion, or internal fault. Return a stable error code and a safe user message; put sanitized diagnostic detail in the restricted log with a correlation ID.

Validate configuration before reading source data. Reject unknown fields if they would hide a typo. Require explicit environment selection and approved destinations. A development default must not silently select production.

For nondeterministic models, record seed and generation settings where supported, but do not promise exact reproduction merely because a seed exists. Runtime versions, hardware, endpoint behavior, and model changes can affect outputs.

12 | Data pipelines and data contracts

12.1 The first reliable pipeline

Start with a versioned command, an explicit configuration file, a scoped identity and a run manifest. A human can invoke it during development. Add a scheduler after repeatability and failure recovery work. Keep dashboard rendering separate from job execution so a browser rerun cannot launch a second expensive pipeline.

Typical stages: acquire approved input; validate; quarantine or reject failures; transform; infer or train; evaluate/validate outputs; publish to an approved destination; record results and notify the authorized operator.

12.2 Dataset contract

Contract element	Example requirement
Ownership	Source department, accountable owner, custodian and escalation
Identity/version	Dataset ID, schema version, extract time, source watermark
Schema	Field names, types, allowed values, units, nullability
Semantics	What a row represents, identifiers, timezone, business meaning
Quality	Duplicate rate, missing required fields, validity and coverage
Delivery	Frequency, late-arrival behavior, maximum expected size
Access	Purpose, principal scope, sensitivity, permitted environment
Change	Notice period, compatibility, consumer acknowledgment
Retention	Source/extract/output retention, deletion and backup treatment

A schema check cannot detect a change in meaning, such as a field switching from gross to net cost while remaining numeric. Include examples and a source-owner change-notification route.

12.3 Pipeline safety and reliability rules

- **Idempotency:** rerunning the same approved input and version must not accidentally duplicate published records. Use input identifiers, run keys, upserts or write-once output versions as appropriate.
- **Atomic publication:** write to a temporary or versioned location, validate, then promote the completed output. Consumers should not see a half-written result.
- **Checkpoints:** record completed stages and watermarks so a retry can resume or safely restart.
- **Retries:** bound attempts and use backoff for transient faults. Do not retry an authorization denial or malformed input indefinitely.
- **Quarantine:** preserve failed-record identifiers and reason codes in an approved restricted location; avoid logging full records.
- **Backfills:** specify the date range, release version, resource budget and duplicate-prevention rule; review downstream impacts.
- **Lineage:** link outputs to input snapshot, code commit, config, model/service version and run ID.

- **Late data:** state whether prior outputs are corrected, superseded or retained; tell consumers how to identify the current version.

12.4 Scheduling and source changes

A systemd timer, cron, or other approved scheduler may be enough for one reliable batch. It needs an owner, overlap prevention, captured exit codes, runtime limits and missed-run detection. Use a workflow orchestrator when multi-step dependencies, backfills and operational visibility justify its maintenance cost.

If a contract breaks, quarantine affected inputs or stop the dependent stage, preserve evidence and notify the owner through the approved route. Do not silently coerce incompatible data just to make the dashboard green. Update tests and the contract before resuming.

12.5 Run manifest example

```
{
  "run_id": "demo-run-20260909-001",
  "product_id": "AI-001",
  "release_id": "0.1.0",
  "code_commit": "<immutable commit>",
  "input_snapshot": "DS-101/snapshot-007",
  "config_digest": "<sha256>",
  "model_service_version": "<approved service revision>",
  "started_at": "2026-09-09T10:00:00Z",
  "status": "completed",
  "records_in": 100,
  "records_out": 97,
  "records_quarantined": 3,
  "output_manifest": "<approved artifact reference>"
}
```

Include duration, schema versions, quality results, resource usage, error categories and relevant evaluation references as available. Keep tokens, raw prompts, protected text and credentials out of the general manifest.

13 | Models, evaluation, and reproducibility

13.1 Start with the simplest credible baseline

An AI use case may be solved adequately by search, rules, statistics, a conventional model, or workflow improvement. Compare a candidate against the current process and a simple technical baseline. Choose complexity only when measured value supports its additional data, compute, review and maintenance burden.

Separate training, development/tuning and final evaluation data. Choose splits that reflect deployment: by time for forecasting, by person/customer/document family where related records could leak information, and by source when testing transfer. Fit preprocessing on training data and apply it consistently to held-out data.

13.2 Match evidence to the task

Use case	Useful measurements	Common trap
Classification	Precision, recall, confusion matrix, PR-AUC, calibration, subgroup performance	Accuracy can hide a rare but important class
Regression/forecasting	MAE/RMSE, bias, horizon-specific error, interval coverage	Random time-series split leaks future information
Extraction/OCR	Field correctness, missing/extra fields, document-type breakdown, review time	Clean pages hide failures on poor scans
Transcription	WER/CER, diarization error where required, speaker/noise/language slices	One clear audio file is not representative
Retrieval/RAG	Retrieval recall, ranking quality, citation correctness, groundedness, access tests	Fluent answers can hide missing or unauthorized evidence
Summarization/generation	Factuality, task rubric, omissions, unsafe output, human review effort	Model confidence and fluent style are not correctness
Anomaly detection	Useful-alert rate, false positives, missed known cases, review burden	Lack of labels makes recall uncertain
Decision support	Outcome usefulness, human override, error consequences, subgroup effects	Offline score does not establish operational value

Define metric formulas, sampling unit, sample size and acceptance threshold before the final assessment. Keep aggregate results and meaningful slices. Record why the sample represents expected deployment and what it does not cover.

13.3 Evaluation procedure

1. Agree on the task definition, user workflow, unacceptable errors and human-review policy.
2. Assemble permitted representative cases, including difficult and boundary examples.
3. Create trusted labels or a human rubric; document reviewer instructions and disagreements.
4. Reserve a final evaluation set and control access to its answers where needed.
5. Measure the baseline and candidate with identical conditions.

6. Inspect failures by source, input type and affected user groups, not only an overall average.
7. Measure latency, resource use, operational burden and workflow benefit.
8. Record thresholds, uncertainty, limitations and the deployment recommendation.

For generative systems, combine automated checks with calibrated domain-expert review. If an LLM judges outputs, validate the judge against human assessments, record its model/prompt version, and inspect disagreement. Avoid using the same unconstrained model's self-rating as the sole release gate.

13.4 Metrics explained

Precision asks, "Of the items flagged, how many were correct?" Recall asks, "Of the true target items, how many did we find?" WER counts substitutions, deletions and insertions relative to a reference transcript. Latency measures elapsed response time. Drift describes changes in data or behavior; it does not automatically prove accuracy has fallen.

Use uncertainty estimates when decisions depend on a small sample. For example, zero observed critical errors in 100 independent representative trials does not prove a zero error rate; the rough rule-of-three upper 95% bound is about 3%. The approximation's assumptions matter, especially for correlated documents or repeated users. Do not promise universal accuracy from a small pilot.

13.5 Experiment and model records

Record product ID, experiment ID, hypothesis, data snapshot/split, code commit, dependency lock, model and tokenizer revisions, prompt/template revision, settings, seed where relevant, hardware/runtime, metrics, failure examples in approved storage, evaluator, and conclusion.

A simple structured run directory can work initially. An internally hosted experiment tracker becomes useful when comparing many experiments or sharing artifacts. MLflow documents separate tracking, backend storage and artifact-storage components; adopting it also requires authentication, permissions, backups and an operator in your environment. [R11]

Model cards should describe intended use, excluded use, training or provider provenance to the extent known, evaluation data, performance slices, limitations, model/service version, license, access restrictions, owner and review date. Unknown vendor/runtime details should remain explicit unknowns.

14 | Containerized LLM services and shared-model operations

14.1 Treat the model as a dependency with an owner

Your team may control only the application that calls an existing container. Distinguish model weights, inference server/container, API configuration, application prompt, retrieval corpus and user interface. Each can have a different owner and change schedule.

Layer	Record	Typical change authority
Model artifact	Model ID/revision, license, provenance, tokenizer, quantization if known	Model/platform owner
Inference service	Image digest, runtime, accelerator, endpoint, concurrency, limits	Service operator
Client contract	Request/response schema, auth, timeout, errors, streaming	Service owner and application team
Product configuration	Prompt, retrieval, allowed settings, output validation	Product technical owner
Operating policy	Allowed users/data, logs, retention, update notice	Applicable data/security/service owners

Do not assume a model supports a parameter because another vendor supports it. Confirm exposed options such as temperature, maximum output length, structured-output support and context limit. Unavailable controls require a service-owner request or an application design change, not an unauthorized container modification.

14.2 Service-card questions

Ask who operates the endpoint, which model versions are available, how updates are announced, whether version pinning is possible, supported data categories, where inference and logs live, timeout/rate/concurrency limits, maintenance windows, health endpoints, quotas, escalation, and fallback behavior.

Record whether the service is a shared resource. A large batch can degrade interactive customers even on a local network. Coordinate queueing, fair use and load tests with the service owner.

14.3 Adapter contract

Give application code a small interface such as generate(request) -> validated_result. The adapter translates to the approved endpoint, supplies credentials from the approved mechanism, applies timeouts, checks response structure, and returns a stable error category. Keep vendor-specific details inside the adapter.

Set maximum input size and output length, bounded concurrency, cancellation behavior and a per-request correlation ID. A timeout may occur after the server has processed a request; retries can produce duplicates or additional cost. Use request identifiers or deduplication where supported, especially when downstream actions have side effects.

Treat model output as untrusted application input. Validate JSON and field values before use. Do not execute generated SQL, shell commands or tool calls without a separately enforced authorization and validation boundary. For a retrieval assistant, answers should indicate insufficient evidence when appropriate.

14.4 Change and capacity management

Before accepting a model, prompt, embedding, tokenizer, quantization or inference-runtime change, run the relevant frozen evaluations and compare quality, latency, memory and failure behavior. Version the complete evaluated combination. An embedding change can require rebuilding the retrieval index; record compatibility.

Measure sustainable throughput under representative concurrency and input lengths. Record queue wait separately from inference time. Monitor memory exhaustion, context truncation, error rate and downstream timeouts. Avoid publishing a throughput promise based solely on one short prompt.

If using locally cached Hugging Face components, the documented HF_HUB_OFFLINE setting can prevent Hub HTTP calls by that library. It is one library control, not an enclave-wide network guarantee; pre-stage required artifacts and enforce network policy independently. [R12]

14.5 Safe fallback

Define whether the application queues work, presents an unavailable message, falls back to an approved simpler method, or switches to another approved model. Validate each fallback against data and quality requirements. Never route restricted input to an unapproved external service because an internal endpoint is down.

15 | Testing, CI, packaging, and offline releases

15.1 A test strategy the team can maintain

Test layer	What it verifies	Appropriate evidence
Unit/behavior	Important transformation and business rules	Boundary cases and failure outcomes
Contract	Input/output schema and service compatibility	Approved fixtures and schema versions
Integration	Source, model, storage and identity work together	Staging run with scoped permissions
Evaluation	Model outputs are useful and acceptable	Frozen dataset, metric report, expert review
Security	Denied access, output handling, secrets and misuse	Positive and negative authorization tests
Operational	Restart, retry, recovery, load and rollback	Runbook rehearsal with measured results
User acceptance	Customer can complete the intended task	Signed or recorded acceptance and exceptions

Test the behaviors that could produce wrong results, unauthorized access, duplicate work, data loss or failed recovery. Avoid requiring a large collection of tests that merely repeat implementation details. Use synthetic fixtures for routine CI; restrict real evaluation data to its approved environment.

15.2 Minimum automated pipeline

On a merge request, check formatting/static issues using approved tools, validate metadata and Claude.md presence, run meaningful unit/contract tests, and inspect dependency changes. On an approved release candidate, build from the locked environment, run staging/evaluation gates and assemble evidence.

Self-managed runners execute repository-controlled code and can expose credentials or affect other workloads if isolation is weak. Separate untrusted test jobs from privileged release jobs, use narrowly scoped runners/identities, and have the platform owner configure isolation. [R3]

Masked GitLab variables are not a complete defense against malicious job code. Review changes to CI definitions and credential-consuming scripts; apply protected scopes and the approved secret mechanism. [R13]

15.3 Starter GitLab CI template

This example intentionally contains no production deployment job. It assumes an administrator provides a pinned internal Python image, an approved isolated runner tag and dependencies already in the image. Configure those values before adoption. The toolkit contains a matching template.

```
stages:
  - validate
  - test

default:
  image: $APPROVED_PYTHON_IMAGE
  tags:
    - approved-python-runner
```

```
validate_summary:
  stage: validate
  script:
    - python tools/check_summary.py

behavior_tests:
  stage: test
  script:
    - python -m unittest discover -s tests -v
```

The administrator should set APPROVED_PYTHON_IMAGE to a reviewed internal image digest. Validate the YAML with the installed GitLab version before enabling it. UI tests that skip for a missing dependency are useful in a local core-only run but must not satisfy a production UI-test gate.

15.4 Dependency and release bundle

Use requirements.in or equivalent as the editable dependency input. Produce an exact, transitive, platform-appropriate requirements.lock with hashes using the approved resolution process. Include model files and tokenizer artifacts separately if they are required; Python dependencies do not capture them.

```
# On an authorized preparation host, after an exact lock exists:
python -m pip download --only-binary=:all: \
  --require-hashes -r requirements.lock -d wheelhouse

# Inside the target boundary:
python -m pip install --no-index --find-links wheelhouse \
  --require-hashes -r requirements.lock
python -m pip check
```

If a dependency lacks a compatible wheel, build it in the approved target-compatible build environment, review the build inputs, then regenerate and approve the complete artifact manifest/lock. Do not silently drop hash verification or fetch missing files from an uncontrolled source.

The release record includes code tag/commit, image digest, dependency lock hash, model/prompt/config versions, dataset/evaluation references, test results, scan/advisory dates, approvals, runbook and rollback target. A checksum detects mismatches against the manifest; trusted provenance requires the approved custody/signature process as well.

15.5 Protect releases

Separate development and release permissions. Promote the artifact actually tested rather than rebuilding an untracked variant. Preserve release evidence and the previous supported artifact. Keep credentials out of CI artifacts and logs. Use the installed platform's available protected branches, tags, environments and approval controls; document manual controls when an edition lacks an automated gate.

16 | Delivery, acceptance, and support agreements

16.1 Choose the delivery class during discovery

Class	Minimum handoff	Ongoing responsibility
One-time analysis/report	Reviewed output, method, assumptions, provenance, acceptance, retention	Explicit support end and archive owner
Reusable local tool	Code/package, install/run/test guide, sample, limitations, receiver training	Named maintainer or explicit unsupported handoff
Recurring batch	Versioned job, scheduler, access identity, retries, output contract, runbook	Runs, failures, data freshness, renewals and changes
Interactive service/API	Authentication, authorization, deployment, evaluation, usage guide, fallback	Availability, latency, capacity, incidents, updates
Shared model/platform component	Service contract, supported consumers, compatibility, ownership	Change notice, capacity, patching, shared dependency support

A one-time deliverable still needs approved data use, reproducibility, review and an agreed retention decision. It may not need ongoing uptime monitoring. A reusable script may need maintenance even without a server.

16.2 Customer acceptance procedure

1. Agree on acceptance cases and thresholds before the final evaluation.
2. Demonstrate the agreed workflow using approved representative data.
3. Record test results, known limitations, user feedback and unmet conditions.
4. Obtain the business owner's acceptance or a clear return-to-build decision.
5. Obtain the separately required technical, operational and security release decisions.
6. Capture the supported version, scope, support route and next review or closure.

The user interface should state material limitations and required human review in language users understand. Keep implementation details out of ordinary user flows unless those details help the user decide how to use an output.

16.3 Handoff packet

- Product card and receiving-owner acknowledgment.
- Release artifact and immutable manifest.
- README, Claude.md, runbook, user guide and synthetic example.
- Approved data/access references; renewal ownership.
- Evaluation, test and acceptance evidence; known limitations.
- Installation/start/stop commands, configuration schema and dependency bundle.
- Monitoring, incident route, backup/restore and rollback instructions where applicable.
- Training completion, support hours, maintenance allocation and retirement criteria.

Ask the receiving person to perform a supported run without the author's intervention. Record failures in the handoff instructions and fix them before closing. A presentation alone is weak evidence that the product can be operated.

16.4 Support agreement template

Specify who handles user questions, runtime faults, data-source problems and model issues. Define support hours, response targets, escalation, routine maintenance windows, update cadence, backup objectives and funding. Identify exclusions and the process for requesting new features.

Suggested operating modes are best-effort during working hours, scheduled batch support, and business-critical service support. Select targets based on actual staffing; do not promise around-the-clock response from an unstaffed junior team.

16.5 Close the delivery effort

Verify acceptance, receiving/support ownership, open-defect disposition and retention. Update delivery phase to Closed and separately set service state to Monitored, Maintenance only, Transferred or Retired as appropriate. Put remaining enhancements in a new linked project so they do not obscure the accepted release.

17 | Monitoring, health, model changes, and maintenance

17.1 Monitor more than model accuracy

Layer	Measurements	Typical action
Infrastructure	Disk, memory, GPU memory, CPU, certificates, process state	Capacity adjustment or platform incident
Dependency	Source reachability, access expiry, endpoint version, schema	Renew, pause, coordinate owner
Pipeline	Last success, duration, failed/quarantined count, retries	Investigate, replay safely, notify consumer
Data	Freshness, volume, nulls, distribution, coverage	Quarantine or source-owner review
Model	Task metrics, input/output drift, calibration, human disagreement	Investigate and evaluate a candidate update
Product	Adoption, completed tasks, time saved, overrides, complaints	Workflow or scope improvement
Governance	Owner missing, review overdue, stale card, support end	Assign owner, review, transfer or retire

For one-time work, record monitoring as not applicable with the support/retention decision. For live products, missing or stale measurements should never default to healthy.

17.2 Alert contract

Every alert needs a measured signal, threshold, evaluation window, minimum sample size if relevant, severity, owner, route, suppression policy, runbook and resolution condition. Route through approved internal notification systems. Test a synthetic alert before release.

Illustrative batch alert: no successful run by the agreed delivery deadline plus grace period; owner is the batch operator; first action checks scheduler, source access and input arrival; consumers receive a delay notice under the support agreement.

Illustrative quality alert: evaluation metric below its agreed threshold on a representative labeled window with adequate sample size; owner is the technical owner; action is investigation and controlled evaluation. Do not trigger automatic retraining solely from a distribution shift.

17.3 Service measurements

An SLI is a measurement, such as the fraction of eligible requests that succeed. An SLO is its agreed target over a defined window. Define the denominator, exclusions, measurement source and support hours. Keep p50/p95 latency, queue wait, sample count and observation time together.

For a batch, a more useful target may be "approved outputs available by the daily deadline" rather than an API availability percentage. For a quality metric with delayed labels, show the measurement window and age. A reachable endpoint can have unknown model quality.

17.4 Refresh, retrain and reevaluate are different

- **Data refresh:** ingest newer records through an unchanged contract.
- **Index refresh:** update retrieval contents and entitlement mappings.
- **Prompt/configuration change:** alter application instructions or settings.
- **Model update:** change weights, service revision, tokenizer or quantization.
- **Retraining:** fit or fine-tune a model using approved training data.
- **Reevaluation:** measure current or candidate behavior on a trusted set.

Each change has its own cost and authorization needs. Changing the retrieval corpus can change answers even when the model is frozen. Changing a shared endpoint can affect many products; use the dependency inventory to identify consumers.

17.5 Maintenance calendar

Proposed starting cadence: operational checks on each run or service interval; weekly owner review of blockers and product evidence; monthly access/capacity/cost review; periodic restore exercises and lifecycle review proportionate to risk. Follow local requirements if they specify different schedules.

Track access expiry, certificate renewal, package/model patch review, dependency vulnerability-data freshness, source contract review, evaluation due dates, documentation review and support end. Assign a backup and escalation date to each obligation.

18 | Incidents, changes, recovery, and retirement

18.1 Incident response

Trigger: incorrect published results, suspected unauthorized exposure, failed or delayed runs, access revocation, model/service faults, or loss of operational control. **Owner:** the designated incident coordinator under the local response process.

1. Record time, product/release, observed impact and reporter in the approved incident system.
2. Assess severity with the responsible operational/security owners.
3. Contain the impact: pause publishing, disable a feature, stop a job, or use an approved fallback.
4. Preserve permitted evidence without copying sensitive data into broad tickets or chats.
5. Identify affected consumers, records, time windows and dependencies.
6. Recover using the runbook and a known-good approved configuration.
7. Validate access, output correctness and operational health before restoring normal use.
8. Communicate through the assigned channel, then document causes and follow-up owners.

For suspected data exposure, use the organization's security incident route immediately; do not improvise forensic handling or send raw evidence broadly.

18.2 Severity examples

Severity	Example impact	Response pattern
Critical	Suspected serious exposure or consequential harmful outputs	Immediate route under local incident policy; contain
High	Supported service unavailable or materially incorrect outputs	Support owner leads recovery and consumer updates
Medium	Partial degradation with safe workaround	Investigate within agreed support window
Low	Minor defect with little operational impact	Track and schedule normal change

These labels need mapping to your organization's actual incident taxonomy and staffing.

18.3 Change control

Every change has a request, rationale, affected users/dependencies, risk, validation plan, release/rollback plan and owner. Standard low-risk changes follow the normal merge workflow. Material changes to data purpose, access, runtime boundary, model behavior, output consumers or autonomy revisit the relevant review gates.

Emergency changes still require an authorized decision, preserved artifact identity and post-change review. Record what evidence was unavailable, why action was needed, and when deferred validation will occur.

18.4 Backup and restore

Inventory what must be recoverable: source code, registry database, configurations, approved model artifacts, output manifests, access references, documentation and audit records. Treat raw-data backups according to source-owner retention and copying rules.

Agree on RPO (how much recent data loss is acceptable) and RTO (how long recovery may take). Encrypt and restrict backups as required; keep necessary keys in the approved mechanism. Test restoration to an isolated environment and verify application behavior, data integrity and permissions.

A SQLite file copied mid-write may be inconsistent unless the chosen backup method accounts for active transactions and journals. The production plan should use the database's supported backup method and prove restore. A Git remote is useful history but does not back up all data, settings and service configuration.

18.5 Rollback procedure

Identify the approved prior artifact, check database/schema and data compatibility, preserve incident evidence, stop or drain active work, apply the rollback, run smoke and permission tests, then restore traffic or schedules. Record the result and affected outputs.

A code rollback cannot automatically undo source-data writes, data deletion, schema migration or permission changes. Use backward-compatible migrations and explicit compensation/reprocessing plans where needed.

18.6 Retirement procedure

Confirm that the business owner approves retirement and affected consumers have a replacement or accepted end date. Stop schedules and endpoints, remove routing, revoke unused service identities and secrets, close alerts, and dispose of or archive records under the agreed retention policy. Handle indexes, caches, extracts and backups explicitly.

Preserve the final product card, released version, acceptance/retirement decision, lineage and required evidence. Mark service state Retired and record a responsible archive owner. Remove the product from active capacity planning while keeping searchable history at the permitted visibility.

19 | Team training, cadence, capacity, and lead responsibilities

19.1 A practical learning progression

Stage	Supervised work	Evidence of readiness
First week	Clone, environment setup, synthetic run, simple issue and merge request	Teammate reproduces a documented run
Weeks 2-3	Input checks, clear function, test, documentation update	Reviewed change with relevant failure cases
Weeks 3-6	Bounded pipeline/model integration and evaluation	Explains data flow, metrics and limitations
After demonstrated readiness	Own a small deliverable with a deputy	Accepted handoff and complete product record
Operational responsibility	Shadow incidents, releases and restores	Performs a runbook rehearsal under supervision

Progress by demonstrated capability rather than a fixed calendar. Give each junior contributor a reviewer and a bounded task. Rotate review and documentation work so knowledge does not accumulate only with the lead.

19.2 Working cadence

- **Daily or asynchronous:** owner, next action, blocker and help needed; update the work item.
- **Weekly delivery review:** examine work in progress, blocker age, imminent decisions and customer feedback.
- **Weekly technical clinic:** discuss one implementation, test, evaluation failure or operational lesson.
- **Regular customer checkpoint:** demonstrate completed behavior and resolve scope questions.
- **Monthly portfolio/operations review:** confirm ownership, capacity, support burden, value and renewal obligations.
- **Periodic process review:** remove duplicate records, improve templates and audit a small sample of products.

Use meeting time for decisions and obstacles, not reading every status aloud. The dashboard should make routine status visible beforehand.

19.3 Capacity and work-in-progress policy

Estimate capacity after leave, meetings, reviews, onboarding, support and platform work. Allocate time for maintaining existing products before promising new builds. A starting planning split might reserve 20-30% for support, reviews and shared improvement; measure actual demand and revise it.

Limit concurrent work so contributors can finish and reviewers can keep up. A practical initial rule is one primary build task per contributor plus an explicit support rotation. The lead should track the review queue as carefully as the development queue.

Escalate blocked access early, but let staff work on independent synthetic interfaces, documentation or approved tests while waiting. Do not let blocked work disappear from estimates or become unauthorized data work.

19.4 Lead data scientist's recurring questions

1. Which products have no effective owner, deputy or funded support?
2. Which estimates rely on unverified data, labels or model capabilities?
3. Where are access and platform waits consuming calendar time?
4. Which live products have unknown quality or stale monitoring?
5. Are repeated components worth maintaining once as a shared package?
6. Can someone other than the author reproduce and support each product?
7. Are we accepting more work than the review and operational capacity allows?
8. Which products should be simplified, transferred, paused or retired?
9. Are customers realizing the intended benefit after acceptance?
10. Which controls prevent actual mistakes, and which create duplicate paperwork?

19.5 Reuse without uncontrolled sharing

Create shared adapters, validation helpers, evaluation utilities, packaging patterns and templates when at least a concrete second consumer exists. Give shared code an owner, version, tests and compatibility policy. Separate reusable code from department-specific data, credentials and assumptions.

Maintain a skills matrix covering Git, Python packaging, SQL/data access, testing, evaluation, container operation, security handling and customer discovery. Use it to assign support and training, not as a substitute for observed work quality.

20 | Standard operating procedure catalog

Use these as short execution checklists. Each has a trigger, owner, ordered actions and completion evidence. The detailed chapters explain design choices. Store the adopted versions in Confluence, link version-specific commands in Git, and assign a procedure owner and review date.

SOP-01 | Register and qualify a request

Trigger/owner: new customer request; portfolio coordinator.

1. Assign a product/request ID and collect department, sponsor and POC.
2. Conduct the ranked interview in Chapter 4 using permitted examples.
3. Record the problem, value, initial scope, sources and constraints.
4. Mark each answer confirmed, estimated or unknown; assign unresolved actions.
5. Route the discovery packet to the technical owner and priority team.

Complete when: the request is discoverable, owned, and has a next decision. **If blocked:** return the specific unanswered questions to the POC; record the wait.

SOP-02 | Run a feasibility investigation

Trigger/owner: a material unknown prevents planning; technical owner.

1. Define one answerable feasibility question, time budget and stop criterion.
2. Obtain permitted representative samples or document why none are available.
3. Measure a simple baseline and the candidate approach.
4. Record failures, dependencies, confidence and follow-on effort range.
5. Recommend proceed, rescope, investigate further or stop.

Complete when: evidence supports the next decision; the experiment is reproducible. **If blocked:** do not present synthetic-data success as real-data feasibility.

SOP-03 | Request and verify data access

Trigger/owner: new data purpose, identity or scope; technical owner coordinates.

1. Identify owner/custodian and minimal scope.
2. Record allowed processing, environment, retention and derivatives.
3. Obtain required decisions in the department's access system.
4. Have administrators provision the approved identity and connection.
5. Test allowed and denied operations from the real runtime.
6. Record verification, expiry and renewal owner.

Complete when: approved and verified access exists for the intended workload. **If blocked:** assign a departmental action owner; never borrow credentials.

SOP-04 | Onboard a team member

Trigger/owner: new contributor; lead and assigned buddy.

1. Obtain individually assigned accounts and necessary approved access.
2. Explain handling rules, incident route and repository workflow.
3. Read the product card, README and Claude.md.
4. Reproduce a synthetic example and run a test.
5. Complete a small reviewed merge request.

Complete when: the contributor can explain the code path and obtain review. **If blocked:** capture missing environment or documentation steps in the onboarding issue.

SOP-05 | Start a project repository

Trigger/owner: approved delivery effort; technical owner.

1. Use the adopted repository template and assign a deputy.
2. Create README, Claude.md, loader, product reference and issue templates.
3. Configure main-branch/merge permissions and review procedure.
4. Add a synthetic example, input schema and meaningful test.
5. Record the internal dependency and execution environment.

Complete when: another person can clone, test and locate ownership. **If blocked:** fix foundational gaps before expanding model code.

SOP-06 | Develop and review a change

Trigger/owner: ready work item; contributor, with independent reviewer.

1. Create a short-lived branch and implement the bounded behavior.
2. Review data handling, errors, dependencies and interfaces.
3. Run the relevant tests and evaluation.
4. Update Claude.md and affected instructions or record a no-change rationale.
5. Open the merge request with evidence; address review.

Complete when: authorized merge and linked evidence are recorded. **If blocked:** split oversized work or obtain a qualified reviewer.

SOP-07 | Use a coding assistant

Trigger/owner: approved AI-assisted task; contributor remains accountable.

1. Verify the specific tool/endpoint and data-use approval.
2. Provide Claude.md, a bounded task and synthetic/approved context.
3. Inspect proposed commands, packages and file changes.
4. Run checks and explain the resulting behavior.
5. Obtain human review and keep the summary accurate.

Complete when: the result meets the same quality gate as other code. **If blocked:** use manual implementation or a smaller task; do not change data boundaries.

SOP-08 | Import packages, images or model artifacts

Trigger/owner: a new dependency/version; platform/import owner.

1. Record provenance, license, versions, platform and intended use.
2. Resolve a complete compatible bundle in an authorized environment.
3. Capture inventory, checksums, scans and required approvals.
4. Transfer through the approved route; verify inside the boundary.
5. Test in staging and promote the same approved artifacts.

Complete when: the target installs/runs without uncontrolled downloads. **If blocked:** retain the existing supported version or request the missing artifact.

SOP-09 | Ingest or refresh data

Trigger/owner: approved input arrival or schedule; pipeline owner.

1. Verify current access and source contract.
2. Assign a run ID and record the input snapshot/watermark.
3. Validate schema and quality before transformations.
4. Quarantine failures under the approved handling rule.
5. Publish validated outputs atomically and write the manifest.

Complete when: outputs, counts, lineage and failure records reconcile. **If blocked:** stop affected publication and contact the source owner.

SOP-10 | Evaluate a model or prompt

Trigger/owner: candidate behavior change; technical owner and domain evaluator.

1. Confirm use case, metrics, sample coverage and acceptance rules.
2. Use the reserved evaluation set and versioned rubric.
3. Compare baseline and candidate under equal conditions.
4. Inspect difficult cases, uncertainty and performance slices.
5. Record quality, latency, limitations and release recommendation.

Complete when: the decision is supported by reproducible evidence. **If blocked:** label quality unknown and obtain missing expert judgments.

SOP-11 | Conduct customer acceptance

Trigger/owner: technically validated candidate; business owner.

1. Demonstrate the agreed workflow and acceptance cases.
2. Have intended users perform representative tasks.
3. Record results, limitations and change requests.
4. Obtain acceptance, conditional acceptance or return-to-build.
5. Separate new scope from defects in the agreed scope.

Complete when: an authorized customer decision exists. **If blocked:** retain Validation status and a dated next action.

SOP-12 | Release a maintained product

Trigger/owner: G4 candidate; designated release authority/operator.

1. Confirm approvals, exact artifacts, support owner and current access.
2. Rehearse installation, smoke tests and rollback in staging.
3. Verify monitors, alert routes, backups and runbook.
4. Deploy the evaluated version during the agreed window.
5. Check behavior and permissions, then record the deployment.

Complete when: supported operation and owner acknowledgment are established. **If blocked:** keep the prior supported version or remain not live.

SOP-13 | Deliver and close one-time work

Trigger/owner: accepted report or finite analysis; technical and business owners.

1. Deliver the reviewed artifact, method, limitations and provenance.
2. Confirm allowed audience and retention.
3. Reproduce the result from recorded inputs/configuration.
4. Agree on support end, archive owner and receiving responsibility.
5. Close delivery and set the correct service state.

Complete when: acceptance and residual obligations are explicit. **If blocked:** resolve receiving ownership before closure.

SOP-14 | Operate and review health

Trigger/owner: each run/service interval and planned review; operations owner.

1. Inspect recent runs, source freshness, endpoint health and access expiry.
2. Check quality evidence age separately from uptime.
3. Follow alerts with a named owner and runbook action.
4. Update health using current evidence; use stale/unknown when warranted.
5. Review capacity and upcoming maintenance.

Complete when: failures have actions and healthy claims have current evidence.

SOP-15 | Renew or revoke access

Trigger/owner: review date, expiry, departure or purpose change; access owner.

1. Verify the continued business purpose and consumer list.
2. Request renewal or revocation from the authorized department.
3. Have administrators update entitlements and secrets as applicable.
4. Test the resulting access, including denied scope.
5. Apply the derivative/cache handling policy and update the registry.

Complete when: effective permissions match the decision. **If blocked:** pause dependent reads after expiry under the local policy.

SOP-16 | Respond to an incident

Trigger/owner: fault, harmful result or suspected exposure; incident coordinator.

1. Record impact and severity through the approved route.
2. Contain unsafe output or affected processing.
3. Preserve permitted evidence and identify consumers.
4. Recover from a known-good configuration.
5. Validate, communicate and assign follow-up actions.

Complete when: service/output safety is restored and ownership of follow-up is clear. **If blocked:** escalate within the incident process.

SOP-17 | Update a model, prompt or index

Trigger/owner: desired change or service-owner notice; technical owner.

1. Identify impacted products and data/permission changes.
2. Version the candidate and record compatibility.
3. Run regression, quality, performance and access tests.
4. Obtain affected approvals and use controlled release.
5. Observe the new version and retain a valid rollback route.

Complete when: the approved version and measured behavior are recorded. **If blocked:** keep the supported version or suspend incompatible use.

SOP-18 | Restore or roll back

Trigger/owner: recovery exercise or incident; operations owner.

1. Identify the recovery target and compatibility constraints.
2. Use approved backups/artifacts in an isolated or controlled environment.
3. Verify integrity, application behavior and permissions.
4. Resume traffic/jobs only after validation.
5. Record achieved recovery time and any data loss.

Complete when: recovery objectives and evidence are assessed. **If blocked:** invoke the continuity plan; do not improvise data-destructive steps.

SOP-19 | Manage a material change

Trigger/owner: scope, users, data, automation or runtime changes; technical owner.

1. Record rationale, impact, risk and affected consumers.
2. Revisit relevant gates and estimate added effort/support.
3. Obtain the proper business/data/security/platform decisions.
4. Validate, release and update cards, runbooks and Claude.md.
5. Record post-change observation and remaining work.

Complete when: behavior and authorization match the documented decision.

SOP-20 | Transfer ownership or offboard

Trigger/owner: staff/team change; lead and receiving owner.

1. Inventory products, access, schedules, secrets and support obligations.
2. Have the receiving owner perform a runbook rehearsal.
3. Record acceptance, primary/deputy and escalation route.
4. Update source permissions and rotate/revoke credentials as required.
5. Update product cards and directories.

Complete when: no product or active identity is orphaned.

SOP-21 | Retire a product

Trigger/owner: approved end of service; business and operations owners.

1. Confirm consumer notice, replacement and end date.
2. Stop jobs/endpoints and remove routing/alerts.
3. Revoke unused identities and handle derivatives/backups.
4. Archive required evidence or dispose under retention rules.
5. Mark Retired and name the archive owner.

Complete when: no unowned processing remains and records are handled.

SOP-22 | Review the portfolio

Trigger/owner: agreed review cadence; portfolio coordinator and lead.

1. Check missing owners, stale cards, blockers and overdue decisions.
2. Compare commitments to delivery/review/support capacity.
3. Review value evidence and lifecycle obligations.
4. Send comparable changes to the prioritization team.
5. Record decisions, next actions and dates.

Complete when: active cases have a clear next decision and accountable owner.

SOP-23 | Record an exception

Trigger/owner: a required control cannot currently be met; control owner.

1. Describe the exact unmet requirement and affected scope.
2. Record risk, compensating measures, owner and expiry.
3. Obtain the authorized exception decision.
4. Track remediation and prevent the exception from silently expanding.
5. Close or renew through the same authority.

Complete when: the exception is explicit, bounded and reviewable. The team cannot authorize exceptions outside its authority.

SOP-24 | Publish an approved metadata projection

Trigger/owner: dashboard/report integration; registry and data/security owners.

1. Define the allowed audience, fields and aggregation level.
2. Exclude secrets, raw data and restricted project details.
3. Apply authorization before rows, counts, search and exports.
4. Include synchronization time and mark stale results.
5. Test unauthorized users and revoke access in a test scenario.

Complete when: the dashboard exposes only the approved projection. Source access and product visibility remain independent.

21 | Copyable templates and required records

The companion toolkit includes editable forms derived from this chapter. Keep one canonical copy of a field; use IDs and links elsewhere. "TBD" is acceptable only with an owner and a decision date, and must not pass a gate that requires the answer.

21.1 Product card

```
schema_version: 1
product_id: AI-000
title: "<short capability name>"
requesting_department: "<department>"
business_owner: "<named accountable person>"
customer_poc: "<daily contact>"
technical_owner: "<named person>"
technical_deputy: "<named person>"
operations_owner: "<person/team or explicit handoff decision>"
problem: "<current problem>"
baseline: "<measured current process>"
value_hypothesis: "<benefit and evidence>"
mvp: ["<required capability>"]
out_of_scope: ["<excluded capability>"]
delivery_type: "One-time analysis"
delivery_phase: Intake
service_state: "Not live"
priority:
  value: Unranked
  authority: "<prioritization team>"
  decision_ref: null
data_refs: ["DS-000"]
access_request_refs: ["AR-000"]
repository_ref: "<approved repository>"
confluence_ref: "<canonical narrative page>"
acceptance_ref: "<criteria and decision>"
support_and_retention_ref: "<agreed policy>"
next_action: "<action>"
next_action_owner: "<person>"
next_review_date: "YYYY-MM-DD"
evidence_status: unknown
```

21.2 Data/access register

```
request_id: AR-000
product_id: AI-000
dataset_id: DS-000
source_department: "<department>"
data_owner: "<authority>"
custodian: "<technical POC>"
purpose: "<specific permitted use>"
principal_ref: "<human/group/service reference>"
environment: "<approved runtime>"
scope: "<rows/fields/operations>"
handling_label: "<local marking>"
approval_ref: "<authoritative decision>"
status: Requested
provisioned_at: null
verification_ref: null
expires_at: null
renewal_owner: "<person/team>"
credential_reference: "<restricted secret-store reference if needed>"
derivative_and_retention_rules: "<approved conditions>"
```

Store credential references only at the permitted visibility. No template field authorizes inclusion of a secret value.

21.3 Development plan and work item

Plan: product/project IDs; problem; users; sponsor/POC; scope/exclusions; baseline; acceptance; data/access; architecture; work packages; owners; estimates; external waits; evaluation; security; delivery; support; risks; decision dates.

Work item: issue ID; product/project; observable outcome; context; acceptance conditions; assigned owner/reviewer; prerequisites; input/output contract; estimated effort; evidence link; documentation impact; completion decision.

21.4 Evaluation and model/service card

Evaluation record: run ID; hypothesis; baseline and candidate versions; data snapshot; split strategy; sample coverage/count; label/rubric version; metrics and thresholds; uncertainty; subgroup/failure analysis; latency/resources; access/misuse results; evaluator; recommendation; limitations.

Model/service card: owner/deputy; endpoint reference; inference location; approved data types; model/revision; image/runtime; tokenizer/quantization if known; request/response contract; context/output limits; concurrency/timeouts; authentication route; logging/retention; update notice; support hours; consumers; next review.

21.5 Release and runbook

Release manifest: release ID; code commit; artifact/image digests; lock/config/prompt/model versions; data/index versions; evaluation results; approvals; deployment environment; operator/time; smoke test; rollback target; known issues.

Runbook: product and version; owner/deputy; purpose; prerequisites and access references; start/stop/run commands; expected outputs; health checks; known faults; restart/retry/backfill rules; escalation; restore/rollback; maintenance and retention; last rehearsal.

21.6 Decisions, risks and exceptions

Architecture decision: ID/date; context; options considered; selected option; rationale; security/data implications; costs; consequences; owner; revisit trigger.

Risk/dependency: ID; description; affected milestone; probability/consequence; owner; mitigation; needed decision; due date; external wait; status; evidence.

Exception: control/requirement; reason; affected products/users/data; risk; compensating controls; authority; approval reference; owner; expiry; remediation plan; closure evidence.

21.7 Incident and retirement

Incident: ID; product/release; reporter/time; severity; impact and affected outputs; containment; timeline; evidence references; recovery; validation; customer/security communications; cause analysis; actions/owners/dates.

Retirement: product/version; approving owner; rationale; consumer notice; replacement; end date; disabled jobs/endpoints; revoked identities; derivative/backup disposition; retained evidence; archive owner; final verification.

21.8 Evidence quality and review

For every estimate or important claim, record confirmed, estimated or unknown; include who supplied it, when, the supporting reference, and the next validation action. Add created/updated/version fields to durable records. An automated presence check can detect a missing form; an accountable reviewer must judge whether its contents are correct.

22 | Dashboard A: project and product portfolio

22.1 Purpose and intended audience

This dashboard answers: what are we building, who requested it, who owns it, what is blocked, what has been delivered, and what remains supported? Its main audience is the lead, contributors, approved customer representatives and the prioritization team. Each audience sees only its authorized metadata.

Start with a registry view that reduces manual status gathering. Do not begin by embedding an LLM into the dashboard; the status problem is primarily structured information, ownership and consistent updates.

AI PORTFOLIO SYNTHETIC EXAMPLE

5	2	2	2
Products	In delivery	Monitored	Blocked

Product	Delivery	Service	Next action
AI-001 Document extraction	Build	Not live	Verify access
AI-002 Policy search	Delivered	Monitored	Quality review
AI-003 Historic trend	Closed	Transferred	Archive
AI-004 Audio evaluation	Discovery	Not live	Obtain labels
AI-005 Anomaly queue	Delivered	Monitored	Investigate run

Filter by department, owner and phase. Open a card for decisions, effort and history.

The figure is a proposed screen layout using synthetic records. The companion Streamlit app implements the core filters, metrics, product cards, local demo edits and audit history. It is a reference implementation, not an organizationally deployed service.

22.2 Screen requirements

Screen area	Content	Behavior
Summary	Active delivery, monitored products, blocked cases, overdue reviews	Counts respect authorization and current filters
Filters	Department, owner, delivery phase, service state, priority, blocked	Define "all" and empty-result behavior
Portfolio table	ID, title, owner, lifecycle, blocker, priority, next review	Stable IDs; clear sort and freshness
Product detail	Scope, people, data/access references, links, effort, decisions	Authorized fields only
Action queue	Next action, owner, due date, external dependency	Route to responsible team; no automatic reprioritization
Change history	Who changed which fields, when and why	Append-only external audit in production

Use text labels with colors so state is understandable without color perception. Show last synchronized and last owner-verified timestamps. Distinguish an unranked case from a case assigned the lowest priority.

22.3 Metric definitions

- **In delivery:** projects in Discovery, Ready, Build or Validation; decide separately whether paused projects remain in capacity counts.
- **Monitored:** products with service state Monitored and an assigned operations owner.
- **Blocked:** active work with an unresolved blocker; show blocker category and age.
- **Review overdue:** next review before today's date and not completed/waived through the defined process.
- **Effort committed:** remaining engineering work, separate from elapsed wait and recurring support.
- **Evidence freshness:** elapsed time since the source measurement or owner verification.

The demo's simpler counts are defined in registry.py. Production formulas should be versioned, documented and reviewed so two dashboard pages do not report different meanings of "active."

22.4 Build steps

1. Agree on the entities and lifecycle values in Chapters 3 and 5.
2. Choose the current authoritative registry: reviewed metadata files initially, database when controlled editing is needed.
3. Load a synthetic dataset and build the table, filters, counts and detail view.
4. Add validated forms with required ownership, allowed transitions and an explicit reason for changes.
5. Add concurrency control and atomic audit recording before enabling multi-user writes.
6. Integrate identity and authorization in the backend; do not rely on hidden UI controls.
7. Add read-only GitLab/Confluence synchronization for selected fields and links.
8. Test role visibility, counts, searches, edits, exports, stale data and recovery.
9. Pilot with approved metadata, train owners, and measure update completeness.

22.5 Production acceptance criteria

A customer from Department A cannot retrieve Department B's restricted rows through the page, an API request, search, counts, an export, a cached result or a guessed record ID. A viewer cannot modify a record. A technical owner cannot assign board priority without the relevant permission. A phase transition that requires approval fails when valid evidence is missing.

Two simultaneous edits cannot silently overwrite each other. A failed audit write cannot leave an unaudited business mutation. A stale integration is visibly stale. The app restarts without losing durable records, and a tested restore recovers the registry within the agreed objectives.

22.6 Demo files and commands

The accompanying folder contains portfolio_app.py, registry.py, seed_demo.py, tests, Claude.md and configuration. It uses SQLite for a single-host synthetic demonstration. It does not have SSO, production authorization or source-system connections.

```
# From the extracted ai_team_toolkit directory:
python seed_demo.py
python -m streamlit run portfolio_app.py --server.port 8501
```

Install approved dependencies first using the README's online-preparation/offline-target procedure. Open the localhost address printed by Streamlit on the same machine. The configuration binds to 127.0.0.1. Do not expose this demo on a shared network or add real organizational metadata.

To try a change, open AI-001, expand the demo edit form, select a valid next phase, supply a reason and save. Inspect the audit history. The demo checks state syntax and selected transitions; production must additionally verify real gate evidence and authorization.

23 | Dashboard B: data access and reliability

23.1 Why this is the second dashboard

In your environment, progress can depend more on data owners, credentials and runtime availability than on model coding. A focused access-and-reliability view helps identify who must act and which live products may fail soon. It complements the portfolio rather than duplicating its status table.

The two tabs are **Access readiness** and **Service evidence**. They share product and dataset IDs with the portfolio, but use more restricted detail permissions.

ACCESS & RELIABILITY SYNTHETIC EXAMPLE

3	1	1	2
Access not effective	Renewal within 7 days	Failed check	Stale / unknown

Access readiness

AR-001 Approved; not yet verified	IAM team: provision and test
AR-002 Verified; expires in 5 days	Owner: renew entitlement
AR-004 Expired	Operator: pause affected reads

Service evidence

Endpoint: healthy | Evaluation: stale | Nightly job: failed

Evidence timestamps and responsible owners are visible. No credential values.

23.2 Access-readiness view

Field/indicator	Meaning	Action
Requested / approved / provisioned / verified	Separate milestones	Route to the next responsible authority
Effective now	Verified grant with required conditions still valid	Allow only the actual source policy to govern access
Waiting age	Elapsed time awaiting needed readiness	Escalate an overdue dependency
Expiry and renewal	Upcoming or expired entitlement	Assign renewal owner and date
Blocker owner	Person/team that can advance the next step	Lead coordinates; dashboard never grants access
Affected products	Products dependent on the same grant/source	Assess impact and resequence work

Store human and service principals as references, never secret values. Limit visibility of principal/group names and detailed source paths if those are sensitive. The production view should distinguish grant expiry from a pending request's service-level deadline.

The example's effective-access function requires a Verified status, a non-expired date and a completed verification date. Real systems may require additional conditions such as current group membership, approved purpose, environment, clearance/attribute requirements, synchronized entitlements and network policy. The dashboard's status is evidence about access, not the enforcement source.

23.3 Service-evidence view

Show the most recent pipeline result, output-data freshness, model endpoint probe, evaluation date, access/certificate expiry and next owner action. Keep each measurement's source, observed time, maximum acceptable age and evidence reference.

Current evidence	Display	Interpretation
Recent passing check	Healthy	This specific check passed within its window
Recent failed check	Failed or degraded	Follow the check's runbook
Old passing check	Stale	Current condition is not established
No measurement	Unknown	Monitoring or quality evidence is missing
Invalid/future timestamp	Clock/data error	Investigate time and ingestion integrity

Do not show an overall green product merely because the endpoint responds. If aggregating checks, define precedence, required checks, missing-data handling and the evidence window. Show quality and availability separately.

23.4 Integration design

Prefer small collectors that emit a validated, sanitized record: product_id, check_id, metric, value/state, observed_at, window, source, release_id, owner, next_action. Collectors run with only the permissions needed for their task and send data to an authenticated ingestion API.

Avoid giving the Streamlit process unrestricted source credentials. The UI reads permitted metadata through the backend. An access-system collector uses a scoped read-only integration to retrieve approved status fields; it does not retrieve passwords or grant privileges.

Collectors should retry transient delivery failures, deduplicate using event IDs, retain a bounded local queue if approved, and make synchronization failures visible. Missing events must age into stale/unknown status instead of preserving old healthy values indefinitely.

23.5 Build and validation steps

1. Define source contracts and field visibility with source owners.
2. Build the access table and expiry calculation using synthetic records.
3. Add check freshness, failure state and action ownership.
4. Add scheduled collectors only after the metadata contract is stable.
5. Connect internal notification routes with explicit owner approval and test alerts.

6. Test approval-without-provisioning, revoked/expired access, missing events, delayed clocks and source outages.
7. Link incidents and dependency records to affected products.
8. Pilot without automatic remedial actions; add narrowly authorized automation only when justified.

23.6 Run the example

```
# Run seed_demo.py once if the shared demo database does not exist.
python -m streamlit run access_ops_app.py --server.port 8502
```

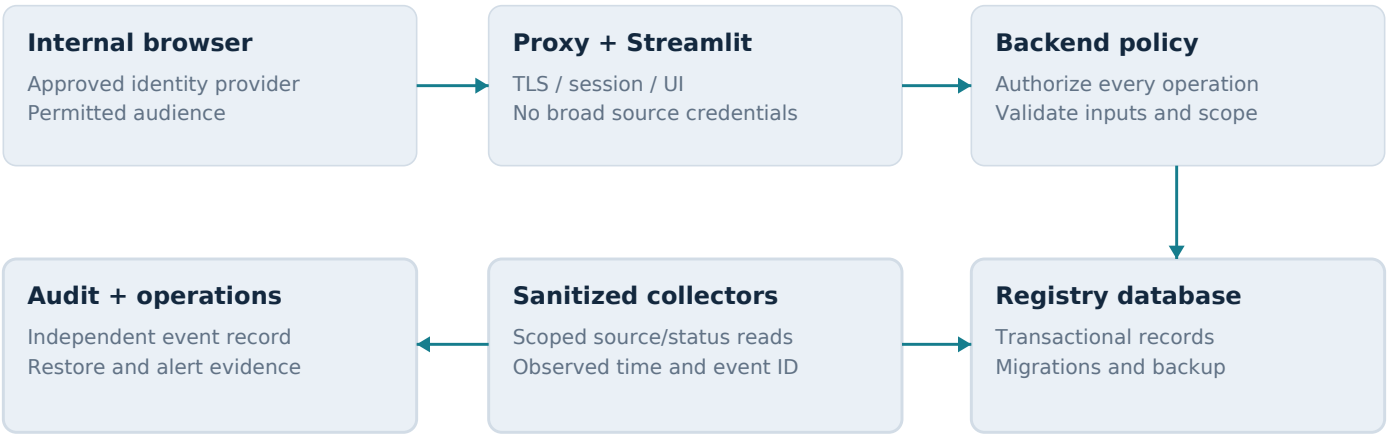
The seeded examples deliberately contain an approval awaiting verification, one expiring grant, one expired grant, a failed nightly job, stale evaluation evidence and missing quality evidence. Dates are relative to the day the database is created. Both apps use the same synthetic SQLite store.

24 | Dashboard engineering plan and production hardening

24.1 Reference architecture

For the educational demo: Streamlit plus a local SQLite file. For a multi-user departmental deployment: internal browser, approved identity provider, reverse proxy/TLS, application backend with authorization, PostgreSQL or approved transactional store, sanitized collectors and an external audit destination.

PRODUCTION PATTERN: AUTHORIZATION BEFORE DATA



The UI is a view; identity, authorization, source access and durable storage have owners.

Streamlit supplies a Python UI. FastAPI is one possible Python backend if the team needs separate authorization, integrations and durable APIs; Flask or Django may also fit local expertise. A separate backend adds work but makes enforcement and testing clearer as multiple interfaces and writers appear. Use the approved installed versions and support model.

24.2 Authentication and authorization

Authentication establishes who the user is. Authorization decides what that user may read or change. Streamlit documents OIDC authentication; application-specific authorization remains a separate design responsibility. The internal identity provider and installed Streamlit version must support the chosen integration. [R6]

Use a validated server-side identity, an authoritative mapping of roles/project membership, and deny-by-default policy checks. Never trust a username typed into a form, URL parameter, unsigned browser value or arbitrary client-supplied header. If a proxy supplies identity headers, strip incoming versions, prevent direct backend bypass and validate the trusted transport.

Apply authorization before database results, counts, search, details, writes and exports. Validate purpose and field-level restrictions where needed. Reject stale/revoked sessions according to the local policy. Keep integration identities separate from interactive users.

24.3 Database schema outline

Table	Important columns	Relationships/controls
products	ID, title, department, business/technical/ops owner, service state	Version, timestamps, restricted visibility
projects	ID, product_id, phase, scope, effort range, next action	Many deliveries per product
memberships	user/group reference, product/department scope, role	Authoritative policy mapping
datasets	ID, owner, custodian, contract, handling label	No secret values
product_datasets	product_id, dataset_id, purpose, contract version	Many-to-many lineage
access_requests/grants	principal_ref, scope, status, approval, expiry	Separate human/service identity, verification
releases	product_id, version, artifact/config/model digests	Immutable evaluated combinations
observations	product_id, check, value/state, observed_at, max_age	Source/event ID and deduplication
decisions/incidents	product/project, type, owner, reference, time	Restricted evidence links
audit_events	actor, operation, object, before/after, reason, time	External append-only sink or equivalent control

Use foreign keys, enum/check constraints, migrations, validated dates, UTC timestamps, and optimistic version checks. Store durations with units. Use a separate connection/transaction per operation as appropriate. Never pass client-supplied SQL fragments into a query; parameterize values and allowlist any dynamic column/sort names.

PostgreSQL row security can provide an additional enforcement layer. Table owners and privileged roles can bypass it unless the relevant controls apply; an application using RLS should not connect as a superuser, table owner or BYPASSRLS role. Design and test database and application policy together. [R8]

24.4 Caching, downloads and privacy

Streamlit's documentation notes that cached values can be shared across users. Do not globally cache unrestricted query results and assume a UI filter will secure them. Prefer no sensitive-result caching initially; any later cache must incorporate validated authorization context, invalidation and revocation behavior. [R7]

Restrict exports, scrub spreadsheet-formula-leading characters if providing CSV where relevant, audit export actions and apply the same row/field policy as on-screen views. Avoid public CDNs, external fonts, telemetry, chat widgets and remote assets in an offline deployment. Do not enable raw-payload debugging in a shared environment.

24.5 Build backlog with acceptance outcomes

Increment	Work	Exit evidence
1: Local prototype	Synthetic schema, two views, filters and cards	Demonstrated user workflow
2: Durable registry	Migrations, validated CRUD, version checks, audit	Concurrent-write and restore tests
3: Identity/policy	Internal login, backend permissions, authorized projections	Negative access tests for every role
4: Integrations	Read-only GitLab/Confluence/access status; sanitized monitors	Sync failures and freshness visible
5: Operations	Container/service setup, TLS, logs, backups, alerts	Runbook and recovery rehearsal
6: Pilot and rollout	Approved metadata, owner training, feedback	Customer acceptance and named operator

Estimate each increment separately using Chapter 4. Unknown identity integration, data visibility rules and hosting ownership are discovery dependencies. Do not extrapolate a two-file demonstration into a production estimate without those answers.

24.6 Deployment and operational checklist

Run under an approved unprivileged identity with restricted filesystem and network access. Mount configuration and secrets through the approved mechanism. Use approved internal images, TLS trust, health checks, resource limits and service supervision. Define how the proxy supports WebSocket traffic for Streamlit. Keep the database on durable managed storage and apply backups/migrations through controlled procedures.

Log actor, action, object ID, outcome and correlation ID without secret values or unnecessary sensitive fields. Restrict the audit destination separately from ordinary editors; the demo's local audit table is not tamper-resistant. Test deployment, restart, revoked access, failed collectors and restored data.

24.7 What the companion code does and does not do

It implements two Streamlit pages, a synthetic SQLite dataset, product filters/cards, validated demo edits, optimistic locking, atomic local audit, access expiry and observation freshness. Tests cover business logic, audit rollback and UI behavior.

It intentionally lacks organizational SSO, real authorization, source connectors, production release approvals, multi-host availability, immutable audit, real notification delivery and infrastructure deployment. These are explicitly planned above. The synthetic marker and loopback setting are safeguards against casual misuse, not substitutes for production security.

25 | Worked examples from request to retirement

25.1 One-time operational analysis

Request: Planning wants an explanation of a historic trend. The customer needs a reviewed report, not a running service.

Discovery: name the sponsor and POC; agree on the question and permitted audience; identify DS-201 and its owner; obtain approved read access; measure the current manual analysis effort. Clarify that the result describes the supplied time window and does not establish causation without supporting design.

Plan: reproduce an approved extract, validate dates/units/missingness, perform the analysis, have a peer review assumptions, and deliver the report plus reproducible code. Create README and Claude.md even if the code is short. Keep data outside Git.

Acceptance: the business owner confirms the report answers the agreed question and receives the limitations. The technical reviewer checks calculations and provenance. Assign the archive owner, support end and retention. Close delivery; set service state Transferred if a recipient accepts responsibility, or use the locally agreed archive/retirement representation.

Optional controls: no uptime monitor or model registry is needed unless a recurring use emerges. If the customer later requests monthly refreshes, create a change project for scheduling, service access, monitoring and ongoing support.

25.2 Recurring document extraction

Request: Operations wants five fields extracted nightly for a human review queue.

Discovery: establish document types, bad-scan frequency, volume, field definitions, error consequences and acceptable review burden. Source DS-101 remains owned by Operations. Request a production service identity separately from the developer's sample access.

Build: validate incoming documents, extract text with an approved parser/OCR stage, call the approved model behind an adapter, validate structured fields, quarantine failures and publish a versioned output. Record input snapshot, parser, prompt, model and release versions. Use deterministic IDs to prevent duplicate publication.

Evaluation: compare against manual review and a simple baseline; measure field accuracy, missed/extra fields and review time across document/scan types. Include malformed files, empty inputs, source schema changes, model timeouts and access denials.

Delivery: the customer accepts the review workflow; the operator rehearses a failed run and safe retry. The product becomes Monitored. Alerts include missed delivery, input freshness, expired access, abnormal quarantine counts and overdue quality evaluation.

Update/retire: a new form layout triggers contract/evaluation review. Retirement stops the schedule, revokes unused identities and handles extracts, outputs and model artifacts under retention rules.

25.3 Internal policy-search assistant

Request: Research wants staff to find answers in approved departmental documents.

Discovery: define authorized audiences, source ownership, document changes, citation expectations, unanswered-question behavior and logging. The organization may already supply an LLM endpoint; verify its allowed data categories and service contract.

Build: ingest permitted documents, preserve source IDs and access metadata, chunk and index, retrieve only currently entitled content, generate an answer with references, and validate structured outputs. Separate document ingestion from interactive answering.

Evaluation: test answer correctness, retrieval recall, citation validity, missing-evidence behavior, prompt injection, malformed content and cross-department access. Include revocation after indexing and after an answer has been cached. Review with domain experts.

Operation: track source/index freshness, entitlement-sync freshness, endpoint health, quality review age, latency and user feedback. Do not automatically log all prompts and answers. Version the model, embedding model, prompt, index and corpus snapshot as needed for traceability.

Update: a model or embedding change requires relevant regression testing; a new source department requires a new data-use/access review. A product can remain Monitored while a separate enhancement project is in Build.

25.4 Decision pattern across the examples

All three products need ownership, approved data, reproducibility, review, Claude.md and acceptance. The recurring batch adds scheduling, retry and delivery-time evidence. The interactive assistant adds per-user authorization, safe retrieval, latency and service operations. Apply controls to actual behavior and consequences rather than the label "AI."

26 | Thirty-, sixty-, and ninety-day adoption plan

The dates below are planning horizons, not a promise of completion. External approvals, platform availability and staffing can change sequencing. Deliver useful foundations while dependencies are being resolved.

Horizon	Lead's priorities	Concrete outcomes
Days 1-10	Inventory people, products, sources, services and constraints	Named owners/deputies, boundary map, missing-access list
Days 11-30	Adopt cards, Git template, Claude.md and intake procedure	Two pilot cases, reviewed repositories, baseline estimates
Days 31-60	Make first pipelines repeatable; establish release/support evidence	One accepted one-time delivery and one operated pilot, if access permits
Days 61-90	Add controlled dashboard pilot and review operational burden	Authorized registry view, measured waits, renewal calendar, improvement backlog

26.1 First ten working days

Meet source owners, platform operators, GitLab/Confluence administrators, security partners and the prioritization team. Ask each for their current service, request route, approval authority, lead times and limitations. Inventory existing work before imposing a new registry.

Audit a small sample of current products: can you identify the owner, source permissions, reproducible run, last accepted version and support responsibility? Capture gaps as work items. Do not ask junior staff to solve missing enterprise ownership by themselves.

26.2 Days eleven through ninety

Days 11-30: run Git and code-review onboarding, select one repository template, publish the customer intake form and require product IDs/Claude.md for new code. Pilot a small one-time use case and a maintained use case. Record actual engineering effort and access wait separately.

Start a manual or Git-reviewed metadata registry. Prototype the dashboards using the supplied synthetic code; do not delay intake improvement until a production dashboard exists.

Days 31-60: establish a repeatable pipeline command, approved dependency bundle, evaluation record and release packet. Assign service identities and operators through the proper owners. Exercise a failure, a retry and a handoff. Build the dashboard's identity and permissions plan before importing real metadata.

Days 61-90: pilot the dashboard with a small approved audience and limited metadata. Add useful read-only integrations, monitor synchronization age and test access revocation. Review which templates are helping decisions, measure support demand and refine capacity commitments.

26.3 Measures of adoption

Track the fraction of active products with owners/deputies, verified access, reproducible commands, current Claude.md, acceptance criteria and support decisions. Measure access-wait days, stale-card rate, review backlog, successful handoff rehearsal and restore success. Use these measures to improve the system; do not reward completing empty forms.

27 | Optional capability menu and selection triggers

This is a menu of additional capabilities to consider as requirements emerge. Each addition needs an owner, approval path, maintenance estimate and removal plan. The simplest approved capability that meets the requirement is usually easier for a developing team to support.

Capability	Add when	Costs/questions to resolve
Workflow orchestrator	Many dependent jobs, backfills or complex retries	Scheduler operations, permissions, metadata, worker isolation
Experiment/model registry	Many comparable experiments and released models	Artifact access, identity, backup, retention, support
Data catalog/lineage service	Sources and consumers exceed manual tracking	Stewardship, sensitive metadata, connector ownership
Data-quality framework	Contracts/checks are repeated across products	Rule ownership, false alarms, versioning
Feature store	Repeated training/serving features need consistency	Time correctness, access, online/offline complexity
Vector/search service	Retrieval needs exceed approved file/relational search	Entitlement filtering, rebuilds, deletion, embeddings
Annotation workflow	Labels require multiple experts and quality control	Reviewer time, restricted UI, disagreement handling
Model gateway	Several approved models need shared routing/quotas	Identity, request policy, logs, fallback approval
GPU scheduling	Shared accelerators face contention	Quotas, isolation, job priority, operator skills
Container orchestration	Deployment/availability complexity justifies it	Administration, upgrades, policy and recovery burden
Canary/shadow releases	Changes need controlled exposure	Parallel costs, privacy, comparison and rollback
Automated drift checks	Stable production baselines and enough observations exist	Statistical power, false alarms, delayed labels
Formal model governance	Consequence or local policy requires stronger oversight	Independent reviewers, evidence and decision authority
Data versioning tools	Reproduction needs exceed snapshot manifests	Access, storage cost, retention and deletion
Internal package service	Many teams reuse approved dependencies	Mirroring/import process, provenance and patching
Knowledge search	Team documentation has become difficult to navigate	Source permissions, freshness, sensitive indexing

Capability	Add when	Costs/questions to resolve
Cost/capacity dashboard	GPU, storage and support burden need allocation	Meter definitions, shared-resource accounting
Multi-tenant hosting	Departments share services with isolation needs	Authorization, resource isolation and audit
Accessibility/localization	User population or policy requires it	Testing, language support, workflow design
Disaster recovery/high availability	Business impact justifies redundancy	RPO/RTO, failover, duplicate dependencies, exercises

27.1 Triggers for extra assessment

Consider additional specialist review for systems that affect employment, health, finances, legal rights, safety, sensitive personal data or other consequential outcomes. The applicable process depends on your organization and use case. This handbook does not establish legal or regulatory compliance.

Autonomous actions, code execution, dynamic SQL, broad tool access and cross-department retrieval add specific authorization and failure risks. Introduce them only with a defined benefit, narrow permissions and tested human/technical controls.

27.2 Avoid early platform overbuilding

Do not adopt every item in this menu. Start with a good repository, reliable command, clear data contracts, measured evaluation, a maintained product card and accountable people. Add shared services when a concrete workload or repeated problem justifies their ownership cost.

28 | Glossary and quick reference

Term	Plain-language meaning
AI / ML	Broad computational approaches; ML learns patterns from data
LLM	A language model that generates or interprets text; outputs require validation
Inference	Running an existing model to obtain an output
Training/fine-tuning	Changing a model's learned parameters using data
RAG	Retrieve relevant documents and provide them to a generator
Embedding	Numeric representation used for similarity/search; may remain sensitive
Baseline	Current process or simple method used for comparison
Ground truth/label	Trusted answer or outcome used to evaluate or train
Pipeline	Repeatable sequence from input through validated output
Data contract	Agreed structure, meaning, quality, access and delivery rules
Lineage	The record of where an output came from and how it was produced
Artifact	A versioned output such as a package, model, report or manifest
Container	A packaged runtime environment for a process
Image digest	Content-based reference to an exact container image
API/endpoint	An interface/address through which software requests a service
Git/merge request	Change history and a request for reviewed integration
CI/CD	Automated checks and controlled delivery/deployment workflows
SSO/OIDC	Sign-in mechanisms used to establish an identity
RBAC / ABAC	Access decisions based on roles / attributes and conditions
Service identity	Account used by a workload rather than a person
Least privilege	Only the permissions required for the approved task
Idempotency	Repeating an operation does not create unintended duplicate effects
Drift	Change in data or model behavior requiring interpretation
SLI / SLO	A service measurement / its agreed target
RPO / RTO	Acceptable recovery data loss / recovery time
SBOM	Inventory of software components in a build
MVP	Smallest useful version that meets an agreed need
POC	Point of contact in this handbook; spell out proof of concept elsewhere
SLA	An agreed service commitment; do not invent one without authority

28.1 Before starting any code

Know the product ID, owner, issue, permitted input, expected output, acceptance condition and execution environment. Read README and Claude.md. Use your own approved identity and a branch.

28.2 Before calling work done

Demonstrate the behavior, show relevant evidence, obtain review, update affected records, and identify the person accepting the result. If the product will keep running, identify who supports it, how failure is detected and what happens when access expires.

28.3 When you are unsure

Record the unknown and its owner. Ask the source owner about data rights, the platform owner about runtime behavior, the customer about usefulness and the technical reviewer about implementation. Escalate to the lead when no responsible owner exists.

29 | Sources, version notes, and deliverable map

29.1 Primary references

References were consulted on 9 September 2026. They support specific technical notes, not an assertion that your internal versions offer every described feature. Use approved offline copies matching installed versions where available.

- **[R1] NIST AI Risk Management Framework.** Voluntary framework organized around Govern, Map, Measure and Manage; useful background for assigning risk responsibilities across the lifecycle. This handbook is a tailored proposal, not an NIST certification checklist. https://www.nist.gov/itl/ai-risk-management-framework
- **[R2] OWASP LLM Prompt Injection Prevention Cheat Sheet.** Input and tool-risk background for LLM application design. https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html
- **[R3] GitLab: Security for self-managed runners.** Execution and credential-isolation considerations. https://docs.gitlab.com/runner/security/
- **[R4] GitLab: Merge request approvals.** Approval behavior and tier-dependent capabilities. https://docs.gitlab.com/user/project/merge_requests/approvals/
- **[R5] pip: Repeatable installs.** Version pinning, hash checking and platform-specific wheel bundles. https://pip.pypa.io/en/stable/topics/repeatable-installs/
- **[R6] Streamlit: User authentication and information.** OIDC sign-in and the distinction from application authorization. https://docs.streamlit.io/develop/concepts/connections/authentication
- **[R7] Streamlit: Caching overview.** Cache scope and shared-state considerations. https://docs.streamlit.io/develop/concepts/architecture/caching
- **[R8] PostgreSQL: Row security policies.** Policy behavior and privileged-role bypass considerations. https://www.postgresql.org/docs/current/ddl-rowsecurity.html
- **[R9] Atlassian: Confluence permissions and restrictions.** Global, space and page permission concepts. https://confluence.atlassian.com/doc/permissions-and-restrictions-139557.html
- **[R10] Claude Code: How Claude remembers your project.** Recognized instruction-file locations and relative imports. https://code.claude.com/docs/en/memory
- **[R11] MLflow: Self-hosting overview.** Tracking, backend and artifact components. https://mlflow.org/docs/latest/self-hosting/
- **[R12] Hugging Face Hub: Environment variables.** Library-level offline behavior. https://huggingface.co/docs/huggingface_hub/en/package_reference/environment_variables
- **[R13] GitLab: CI/CD variables.** Limitations of masking and risks from job code. https://docs.gitlab.com/ci/variables/
- **[R14] Streamlit: AppTest.** Simulated app testing used for the reference UI checks. https://docs.streamlit.io/develop/api-reference/app-testing/st.testing.v1.apptest

29.2 Companion toolkit

Item	Contents
README.md	Setup, offline preparation, commands, boundaries and production path
Claude.md and .claude/CLAUDE.md	Required code summary and recognized loader
portfolio_app.py	Dashboard A: portfolio, cards, filters, demo edits
access_ops_app.py	Dashboard B: access readiness and service evidence
registry.py and seed_demo.py	Validated store, synthetic records, lifecycle/access/freshness logic
tests/	Behavioral, transactional and UI checks
templates/	Editable product, access, development, operational and governance forms
tools/check_summary.py	Summary presence/structure check
requirements files	Dependency inputs and tested version snapshot; approval still required
handbook/	This complete source manuscript and PDF-generation code

29.3 Validation and practical limits

The companion code was exercised with synthetic records using Python and Streamlit 1.55.0. The validation report records exact checks and dependency versions. The tests include successful and rejected edits, audit rollback, concurrency conflict, expired access, stale/missing observations and displayed UI counts/filters.

The PDF is the complete framework and can be used without running the examples. The examples are a starting point for implementation. No internal systems were accessed, no real data or credentials were used, and no organizational deployment or security approval is implied.

Before adopting the framework, assign its process owner, reconcile it with current local controls, select applicable practices, and publish an approved revision. Review the handbook when the team's operating model, internal tools or security boundary changes.